nature computational science

Resource

https://doi.org/10.1038/s43588-026-00991-1

The Quantum Optimization Benchmarking Library

Received: 28 August 2025

Accepted: 15 April 2026

Published online: 23 June 2026

Check for updates

Thorsten Koch[1,2] ✉, David E. Bernal Neira[3], Ying Chen[4], Giorgio Cortiana[5], Daniel J. Egger[6], Raoul Heese[7], Narendra N. Hegade[8], Alejandro Gomez Cadavid[8,9], Rhea Huang[10], Toshinari Itoko[11], Thomas Kleinert[12], Pedro Maciel Xavier[3,13], Naeimeh Mohseni[5], Jhon A. Montanez-Barrera[14], Koji Nakano[15], Giacomo Nannicini[10], Corey O'Meara[5], Justin Pauckert[16], Manuel Proissl[6], Anurag Ramesh[3], Maximilian Schicker[1], Noriaki Shimada[11], Mitsuharu Takeori[11], Víctor Valls[17], David Van Bulck[18], Stefan Woerner[6] ✉ & Christa Zoufal[6] ✉

Recent progress has brought benchmarking of (heuristic) quantum algorithms at scale within reach. Particularly in combinatorial optimization, it is key to empirically analyze and track progress towards quantum advantage. This work introduces a systematic, fair and comparable benchmarking framework for quantum optimization methods by presenting ten model-independent problem classes that are challenging for classical methods. Track records of specific instances and solutions are given in an accompanying open-source repository. While the individual properties of the problem classes vary, they all become challenging from less than 100 to, at most, an order of 100,000 decision variables. We reference results from state-of-the-art solvers for instances across all problem classes and demonstrate exemplary baseline results obtained with quantum solvers for selected problems, which illustrate standardized benchmark reporting. The presented problem instances may be approached with classical or quantum algorithms executed on varying hardware platforms to drive the field towards quantum advantage.

Optimization is subject to extensive research due to its importance in science and industry. Applications include experiment design, protein folding, logistics and finance, where better algorithms can improve performance, accuracy and cost efficiency. Optimization algorithms fall into three categories: provably exact algorithms, provably approximate algorithms with a priori performance guarantees, and heuristic algorithms without such guarantees, although posterior performance bounds may sometimes be available. Complexity theory classifies optimization problems as 'easy' or 'difficult', determining what types of algorithm may exist. Many relevant optimization problems are NP-hard: that is, the computational resources required to find provably optimal solutions grow exponentially with the problem size. We do not expect the existence of provably exact efficient algorithms for such problems in general[1]. For some of them, the possible performance of provably approximate algorithms is limited by known inapproximability bounds[2,3]. Here, inapproximability bounds imply that solving

[1]Zuse Institute Berlin, Berlin, Germany. [2]Technische Universität Berlin, Berlin, Germany. [3]Davidson School of Chemical Engineering, Purdue University, West Lafayette, IN, USA. [4]Departement of Mathematics & Risk Management Institute, National University of Singapore, Singapore, Singapore. [5]E.ON Digital Technology GmbH, Essen, Germany. [6]IBM Quantum, IBM Research Europe—Zurich, Zurich, Switzerland. [7]NTT Data, Tokyo, Japan. [8]Kipu Quantum GmbH, Karlsruhe, Germany. [9]University of the Basque Country UPV/EHU, Leioa, Spain. [10]University of Southern California, Los Angeles, CA, USA. [11]IBM Quantum, IBM Research Tokyo, Tokyo, Japan. [12]Quantagonia GmbH, Bad Homburg, Germany. [13]Federal University of Rio de Janeiro, Rio de Janeiro, Brazil. [14]Forschungszentrum Jülich, Jülich, Germany. [15]Hiroshima University, Higashihiroshima, Japan. [16]T-Systems International GmbH, Frankfurt am Main, Germany. [17]IBM Quantum, IBM Research Europe—Dublin, Dublin, Republic of Ireland. [18]Ghent University, Ghent, Belgium. ✉e-mail: koch@zib.de; wor@zurich.ibm.com; ouf@zurich.ibm.com

a problem better than the given bound is NP-hard, that is, as hard as solving the problem to optimality. In practice, it is often sufficient to find a good solution instead of a provably optimal one. Thus, classic heuristics and machine learning-based solvers are viable approaches for many problems of interest. Since heuristics lack a priori guarantees, rigorous benchmarking is crucial to assess the corresponding performance and resource requirements. To enable a fair comparison of algorithms and hardware platforms and to guarantee reproducibility of reported results, a suitable set of problem instances and metrics needs to be defined. This also allows tracking of progress in algorithm and hardware improvements over time.

Quantum computing offers a novel toolbox to approach optimization problems—see ref. 1 for an in-depth discussion of quantum optimization algorithms and their challenges and potential. With recent advances in quantum hardware[4], systematic benchmarking of quantum optimization algorithms and comparison with classical solvers have become feasible. This Resource proposes ten classes of combinatorial optimization problems and corresponding problem instances, which can be found in the open-source Quantum Optimization Benchmarking Library repository[5]. These problem classes were selected because they include instances that are difficult for state-of-the-art classical solvers with only 100–100,000 variables and have diverse characteristics, and many of them have connections to industrial applications. This makes them suitable candidates for benchmarking and exploring the potential of quantum advantage in optimization. Further, for each class, we provide instances of varying complexity, ranging from those already feasible for today's quantum hardware to those challenging for state-of-the-art classical solvers. Notably, we promote model-independent benchmarking, which allows for maximal flexibility in choosing models, algorithms and hardware platforms. Our goal is to provide a framework to track the progress towards quantum advantage in optimization and have an initial baseline to start with. A prerequisite for this goal is the establishment of a fair and systematic benchmarking effort to advance towards demonstrating quantum advantage in this field. Given the large number of possibilities, exhaustive comparison of algorithms is an enormous challenge that requires the scientific community's collective effort.

There is plenty of literature on benchmarking in general. See, for example, ref. 6 on the LINPACK benchmark used for the TOP500 list, ref. 7 on the SPEC benchmark used to compare hardware performance, ref. 8 regarding benchmarking in optimization, Koch et al.[9] on measuring progress in linear programming/mixed integer programming (LP/MIP) solvers and ref. 10 about Max-Cut and quadratic unconstrained binary optimization (QUBO) heuristics. Many important classical optimization benchmarks have been established via competitions and open-source libraries. Notable examples are the Discrete Mathematics and Theoretical Computer Science (DIMACS) implementation challenges[11], the Satisfiability (SAT) competitions[12,13] and the Mixed Integer Programming Library[14]. Another set of optimization benchmarks is regularly conducted by Hans Mittelmann[15]. Here, different optimization pipelines are executed using the same computational set-up. Most benchmarks in quantum computing focus on the performance of individual components of the quantum computing stack and their integration[16–18]. Randomized benchmarking focuses on gate quality[19]. The layer fidelity represents a quality metric of layers of two-qubit gates[20]. Circuit Layer Operations Per Second measures the speed of a quantum computer[21,22]. Speed and quality of compiling quantum circuits from high-level abstract representations to machine-level instructions have been investigated and reported in Benchpress[21]. In addition, application-driven benchmarks have been proposed as full-stack system benchmarks. They usually focus on problems that can be solved to optimality classically and track the progress of quantum computers to achieve a known optimal solution[23–28]. While this allows tracking of the progress of hardware and algorithms, it is not sufficient to demonstrate quantum advantage, where it is important to choose appropriate benchmarking metrics[29–31].

In addition to identifying the best approach to solving a given problem, benchmarking is critical for verifying algorithms' correctness, identifying bottlenecks, improving existing approaches and tracking progress over time. This includes the progress in algorithms, software libraries and hardware platforms[21]. The goal, scope and limitations of a benchmarking effort must be clearly defined, as this determines the type of conclusion it may support; cf. ref. 28 for a related discussion. The main goals of benchmarking can be categorized as follows. In application benchmarking the goal is to find the best possible algorithms—classical or quantum—to solve a given problem instance. Thus, benchmarks must be model independent to allow all possible approaches to solve a problem. This is the only level that ultimately allows demonstration of quantum advantage. In algorithm benchmarking the goal is to identify suitable strategies for setting hyperparameters, identifying bottlenecks, improving algorithms and tracking progress over time. Since algorithm benchmarking does not entail comparison against all possible algorithms, it will not allow demonstration of quantum advantage. Nevertheless, it can be used to estimate an algorithm's scaling, which may facilitate the identification of potential asymptotic scaling advantages and help track progress towards quantum advantage. In system benchmarking the goal is to identify the best way to run a fixed algorithm for a fixed problem on a given platform. This includes tuning algorithmic hyperparameters or parameters of the execution environment, for example, for error suppression and mitigation, and to confirm that the algorithm is working as expected. It can also be used for application-centric hardware benchmarking.

The set of problem instances presented in this work enables investigation of potential advantages of novel algorithms and computational platforms and allows for a fair and reliable comparison. This is comparable to the curated set of many-body quantum systems presented in ref. 32—but for optimization. Table 1 provides an overview of the problem classes and a summary of their key properties. In particular, it highlights challenges that may be introduced when translating problem instances from integer programming (IP) to QUBO. This includes increasing the number of variables, conversion of sparse to dense problems, or larger ranges of coefficients. This shows the importance of selecting the right modeling approach for a problem. The set of problems was selected to be diverse and can be grouped into three categories.

- Market Split (multidimensional subset sum), Independent Set (unweighted maximum independent set) and Network Design (communications network design problem) are classic binary optimization problems. They were studied intensively before the year 2000. Since then, only limited efforts have been put into improving the state of the art in targeted exact solvers or heuristics. We attribute this to the fact that general MIP solvers and known heuristics are good enough for many practically relevant settings. However, these problem classes are well established, can be effectively solved at small sizes, and become difficult to solve exactly for growing dimensions. The latter is particularly true for Market Split and Network Design. It follows that these problems are great candidates for testing, tracking and pushing the capabilities of quantum algorithms.
- 'LABS (low-autocorrelation binary sequences), Birkhoff (minimum Birkhoff decomposition), Steiner (Steiner tree packing in graphs—very large-scale integration design/wire routing), Sports (sports tournament scheduling) and Topology Design (graph topology design problem) are problem classes that have received limited attention compared with, for example, traveling salesperson (TSP), Independent Set or Knapsack problems. Thus, it may be that the known state-of-the-art classical algorithms could still be much improved. At the same time, quantum algorithms may offer an interesting approach to solving these types of practically relevant problem, especially because LABS and minimum Birkhoff decomposition are already difficult at very small sizes. Steiner tree

https://doi.org/10.1038/s43588-026-00991-1

Table 1 | Problem class overview

Problem	MIP							QUBO			
	NP-hard	Type	Dense	#Vars	Coeffs	Constr	Feas	Bin	Dense	#Vars	Coeffs
Market Split	yes	F/L	d	78	48	1	no	yes	d	70	~5×10^6
LABS	yes	Q/Q	s	81	4	1	yes	yes	s	820	~4×10^4
Birkhoff	yes	L/L	s	240	10^4	3	yes	no	d	3,480	~3×10^{10}
Steiner	yes	L/L	s	423,360	3	9	no	yes	—	—	—
Sports	no	F/L	s	8,608	2	>10	no	no	s	11,791	~4.5×10^3
Portfolio	yes	Q/L	s*	690	~3×10^4	2	yes	yes	d	690	~2×10^9
Independent Set	yes	L/L	s	500	1	1	yes	yes	d	500	2
Network	yes	L/L	s	1,211	10^6	5	yes	no	s	46,330	~2.5×10^{19}
Routing	yes	L/L	s	—	—	<10	yes	no	s	—	—
Topology	no	L/Q	s	2,176	2	4–7	yes	no	s	—	—

Details of illustrative MIP and QUBO formulations, as applicable. 'NP-hard' indicates whether a problem class is NP-hard in its basic formulation. 'Type' denotes the objective/constraint type as linear (L) or quadratic (Q) for either if the problem corresponds to an optimization problem. For feasibility problems, the objective type is indicated as feasibility (F). 'Dense' denotes whether the model is dense (d), defined as $|A|\geq\frac{1}{4}nm$, where $A\in\mathbb{R}^{m\times n}$ is the constraint matrix of an MIP and n, m denote the number of variables and constraints, respectively, or sparse (s), otherwise. For QUBO models, density is defined as $|Q|>\frac{1}{8}n^2$, where we assume that Q is an upper-triangular cost matrix (due to symmetry, hence the reduced prefactor of ⅛ instead of ¼ used for MIP). Again, n denotes the number of variables. Portfolio has a sparse constraints matrix but a dense quadratic objective, which we indicate by s*. '#Vars' denotes the approximate number of decision variables of the models corresponding to instances where the problems can become difficult for classical methods to solve to proven optimality in MIP formulation. Hence, the number of decision variables in the QUBO column denotes the size of the QUBO formulation where solvers struggle to solve the corresponding MIP. This is to highlight how certain problems become larger when represented as QUBO instead of an MIP. 'Coeffs' is the approximate coefficient range, that is, the maximum range of all coefficients in the models. Again, we can have a non-trivial discrepancy between MIP and QUBO representations for certain classes. 'Constr', number of different constraint types in the MIP formulation (≠total number of constraints). This indicates the complexity of modeling the problems. 'Feas' denotes whether it is trivial to find or construct a (initial) feasible solution to the problem or not. For QUBO, all solutions are feasible, and thus we drop the column. 'Bin' indicates whether all variables in the MIP problem are naturally binary or not. For QUBO this is always true and thus not reported. If this is not the case, we see a discrepancy between the number of variables in the MIP and the QUBO models. For LABS, we have natural binary variables; however, we receive quartic terms in the objective from adding squared quadratic equality constraints as penalties that need to be decomposed into quadratic terms by adding binary variables. QUBO instances requiring more than 1.6×10^5 variables have not been generated due to computational restrictions. We indicate this in the respective columns (—). Further, no values are given for Routing problems since we were able to find the optimal solution for all tested instances.

packing and sports scheduling become interesting at larger system sizes, but they have many practically relevant applications and, especially for the latter, even heuristics are struggling.

- Portfolio Optimization (multiperiod portfolio optimization with transaction costs) and Vehicle Routing (capacitated vehicle routing problem—CVRP) problems have obvious practical applications. Companies face diverse variations of these problems depending on industry and operational execution. Hence, there are indefinitely many variations of these problems depending on individual constraints and/or objectives, resulting in problem instances of varying difficulty. For this reason, it is difficult to define a small set of problem instances that are representative of these problem classes to benchmark and compare established approaches. In addition, many established solvers from commercial software vendors are not accessible for open-source benchmarks. Despite the practical relevance of these problem classes, no well-established benchmark set exists for testing the progress of quantum algorithms. We aim to fill this gap by providing a selection of relevant problem instances. However, it should be noted that all given vehicle routing problem (VRP) instances can be solved to optimality with existing methods. This is a shortcoming that we aim to address in the future.

The remainder of this work is structured as follows. In Results, we present benchmark results of the maximum independent set as an illustrative problem. Additional illustrative quantum results—for the LABS and Birkhoff problems—are included in 'Illustrative quantum benchmark' in Supplementary Information. In Methods, we give a high-level overview of all ten problem classes selected. Additional information considering the problem classes such as definitions, and explanations including the relevance for quantum optimization benchmarking, can be found in Supplementary Information.

Results

This work presents ten problem classes and corresponding instances that are non-trivial to solve with state-of-the-art methods—for most of them even at relatively small scales. Furthermore, the respective problem classes are mostly related to practically relevant problems. Such problems are hard to find for two reasons. First, unsolved problems of industrial relevance are rarely published. Second, problems of practical relevance that cannot be solved are usually discarded and replaced by artificial simplifications of the original problem. This leads to a selection bias and may be one reason why a benchmarking framework that includes truly difficult (for reasonable system sizes) and relevant (ideally in practical settings) optimization problems that enable a meaningful and fair comparison of quantum and classical algorithms has not been established until now.

We applied the following criteria for the selection of the problem classes. First, we consider problems that can be modeled using only integer variables, preferably binary ones, where all coefficients can be represented as integers. In an all-integer setting, checking the feasibility of solutions is easy and does not suffer from issues with rounding or numerical accuracy, and integer variables are more amenable for known quantum optimization algorithms without requiring too many qubits. Second, problem classes should contain instances that are hard to solve for state-of-the-art classical approaches, preferably even at a relatively small size, and preferably because of primal difficulty. This enables testing of quantum optimization algorithms already known today. Third, all selected instances should have a feasible solution. For those problem classes that are not genuinely feasible, we generated the instances in a way that guarantees feasibility, as discussed in the following sections.

Figure 1 provides a more detailed overview of the instances provided for the various problem classes, complementing Table 1. For MIP and QUBO formulations of the considered problem instances, it illustrates the number of variables, the problem density, the coefficient range and whether the optimal solution is known. It should be noted that for certain problem instances the optimal solution is known but it is still difficult to compute the solution with state-of-the-art methods from the given model formulation. This is because certain problem instances were created from a solution to guarantee solvability. Please

https://doi.org/10.1038/s43588-026-00991-1

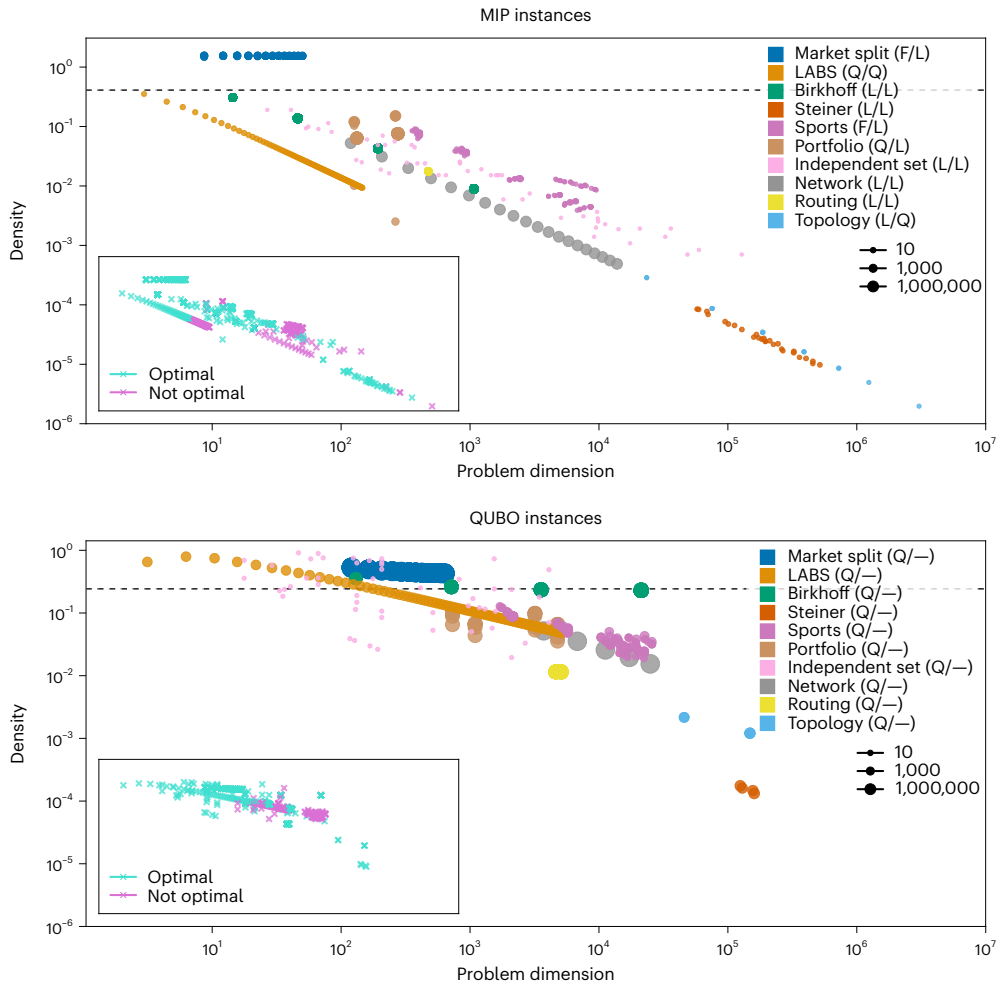

Fig. 1 | MIP and QUBO representations of problem instances. The figures plot the problem dimension (MIP, $\sqrt{\#\text{variables} \times \#\text{constraints}}$; QUBO, #variables) against the density, that is, the fraction of non-zero coefficients in the objective and, for the MIP instances, constraint functions, of the various problem instances provided in the repository[5] for MIP (top) and QUBO (bottom) formulations of the problem. The legend entries for the various problem classes name the class itself, and in brackets give the form of the (objective/constraint) function, with L and Q denoting linear and quadratic, respectively. Furthermore, the size of the individual points is directly related to the coefficient range of the respective problem instance per problem class—see the legend in the center right. The dashed lines illustrate the separation between sparse and dense instances as described in Table 1. Finally, the inset plots illustrate the instances in which the optimal solution is known. It should be noted that, while the MIP plot shows all problem instances, the QUBO plot only shows those instances that we could transform into QUBO models with fewer than 1.6×10^5 variables, cf. Table 1. The respective model translation is done for Birkhoff, Steiner, Sports, Network, Routing and Topology with Qiskit Optimization v.0.6.1 QiskitOpt[165] using the default parameters. Market Split, LABS, Portfolio and Independent Set can be trivially converted to QUBOs using ZIMPL v.3.6.1 Koch[166], since there are no slack variables to consider. Note that for Portfolio this shows the density of the constraint matrix; the quadratic objective is always dense.

note that if multiple MIP and QUBO models are available we choose to represent the one that results in the fewest variables. We would further like to point out that the QUBO illustration only shows those instances that could be converted from an MIP model into a QUBO model with fewer than 1.6×10^5 variables. As the plots show, the larger the number of variables, the more likely it is that we do not know the optimal solution. Finally, it can be seen that Market Split and LABS are the two problem classes for which difficult instances are found at the smallest sizes. Independent Set also has hard instances at relatively small sizes; these instances are also sparse and come with a small coefficient range. For these reasons, these three problem classes seem the most amenable to being approached with a quantum algorithm on near-term hardware.

Next, we are going to illustrate our contributions on the example of one of the ten problem classes: maximum independent set. Details regarding the background and mathematical formulation of the remaining problem classes can be found in 'Problem classes'. The specific instances and respective baseline results are given in Supplementary Information—Problem classes. Furthermore, Supplementary Information—Illustrative quantum benchmarks presents quantum benchmarks for LABS and Birkhoff problems.

Maximum independent set

Background. An independent (or stable) set is a set of vertices not adjacent to each other in a graph, that is, no edge connects any two vertices of the set. The maximum independent set problem (or maximum stable set problem) is to find the largest independent set for a given graph. It is deeply rooted in graph theory and combinatorial optimization and has a long history of research. Its decision problem is one of the first problems shown to be NP-complete[33]. It is known to be strongly NP-hard for general graphs, while it can be solved in polynomial time for certain classes of graphs—for example, P_5-free graphs and perfect graphs. The independent set is a fundamental feature of a graph and relates to various other graph parameters. The maximum independent set for a graph is equivalent to the maximum clique problem for its complement graph. A set of independent vertices is equivalent to its complement set being a vertex cover. Thus, the sum of the size of the

maximum independent set and the size of the minimum vertex cover is equal to the number of vertices in the graph. The maximum independent set is also useful for modeling industry problems—for example, non-intersecting label placement on a map[34], discovering stable genetic components for designing engineered genetic systems[35] and strategic planning, such as choosing locations for stores[36].

There have been extensive studies both on exact and approximate algorithms. As far as we know, the best known exact algorithm can compute the maximum independent set of an n-vertex graph in $1.1996^n n^{O(1)}$ time and polynomial space[37]. As for approximate algorithms, most studies (too many to list here) consider a particular class of graphs and develop algorithms with constant approximation ratios, since the maximum independent set problem, in general, is Poly-APX-complete, meaning it is as hard as any problem that can be approximated to a polynomial factor[38]. Quantum algorithms have also been investigated recently. For example, a method is proposed for solving the maximum independent set problem on unit disk graphs[39] using a quantum simulator with Rydberg atomic arrays[40]. It should be noted that better approximation results are available for unit disk graphs than for the general case[41,42].

Why is it interesting? The unweighted maximum independent set problem is a classic NP-hard mathematical problem. Starting at problem sizes of several hundred variables, there are known instances that are hard to solve to proven optimality with existing methods[43], and which are difficult to tackle even heuristically. An interesting property of this problem class is that it is well suited for translation into relatively sparse QUBOs—often with small coefficients.

Problem statement. Let $G=(V,E)$ be a graph with vertices $V=\{1,\ldots,n\}$ and edges $E\subset V\times V$. The maximum independent set problem for G is

$$\max_{S\subseteq V}|S| \text{ with } i,j\in S\Rightarrow (i,j)\notin E.$$

It can be represented as a binary linear program,

$$\max_{x\in\{0,1\}^n}\sum_{i\in V}x_i \text{ subject to } x_i+x_j\le 1 \text{for all } (i,j)\in E,$$

or as a binary quadratic program,

$$\max_{x\in\{0,1\}^n}\sum_{i\in V}x_i^2 \text{ subject to } x_ix_j=0 \text{for all } (i,j)\in E.$$

Note that $x_i=x_i^2$ holds for binary variables. The constraint in each formulation forbids $x_i=x_j=1$, for all $(i,j)\in E$, that is, both ends of each edge cannot be in an independent set. For the weighted version, some weights $w_i>0, i\in V$, can be added to the vertices.

Instances. Any collection of graphs might be a suitable benchmark set for the maximum independent set. We aimed to collect some instances known to be challenging and added several smaller ones to facilitate progress tracking. We collected instances from various sources, in particular Network Repository[44], SteinLib[45], OEIS implementation challenges[43] and self-generated graphs. Furthermore, we provide a program that takes a graph and an independent set and verifies that it is indeed a stable set in the graph.

Classical baseline. We provide formulations as binary linear programs and as QUBOs. These can be solved by any suitable solver. We provide baseline results for the binary linear program using Gurobi 12.0.1 run on an Intel Xeon Gold 6338 CPU @ 2.00 GHz processor using 128 threads with a timeout of 7,200 s. The baseline results for the QUBO formulation are computed using ABS2 as a heuristic run on four Nvidia A40 48 GB with a timeout of 7,200 s. Corresponding runtime, time-to-solution and absolute gaps to the optimal/best known solutions as well as exact data and instance names can be found in Supplementary Information—Problem classes—Maximum Independent Set.

Quantum benchmark. Next, we outline a benchmark that is based on a standard quantum approximate optimization algorithm (QAOA)[46] on the Independent Set problem instances defined by graphs $G=(V,E)$ with $|V|=17$ and $|V|=52$ variables (Supplementary Information—Illustrative quantum benchmarks). Independent Set is a constrained optimization problem, which we can thus model as a QUBO by adding a penalty term:

$$\max_x \sum_{i\in V}x_i-\lambda\sum_{(i,j)\in E}x_ix_j.$$

Here, λ is a positive Lagrange multiplier to introduce the constraints. We convert this QUBO to a Hamiltonian with the variable change $x_i=(1-z_i)/2$ and promote the spin variables z_i to Pauli-Z spin operators Z_i. The resulting Hamiltonian has two contributions, the objective and the constraints:

$$H_{\text{obj}}=-\frac{1}{2}\sum_{i\in V}Z_i,\qquad \lambda H_{\text{const}}=-\frac{\lambda}{4}\sum_{(i,j)\in E}Z_iZ_j-Z_i-Z_j.$$

We neglect the irrelevant constant energy offsets, that is, terms that are neither linear nor quadratic in Z_i. These terms are $|V|/2$ and $-\lambda|E|/4$ and stem from the objective and constraints, respectively. Therefore, the total cost function Hamiltonian is $H=H_{\text{obj}}+\lambda H_{\text{const}}$. To produce bitstrings from the quantum hardware, we sample from a QAOA trial wavefunction $|\psi(\beta,\gamma)\rangle=\mathrm{e}^{-\mathrm{i}\beta H_{\text{M}}}\mathrm{e}^{-\mathrm{i}\gamma H'}|+\rangle$. Here $H_{\text{M}}=\sum_{i\in V}X_i$ is the standard QAOA mixer. H' is the cost operator $H_{\text{obj}}+\lambda H_{\text{const}}'$ used to design the trial wavefunction, and we apply $p=1$ repetitions. Crucially, H_{const}' may implement only a subset of the constraints in H_{const} to limit the depth of the quantum circuit. We must now optimize the parameters β,γ and λ. In a typical QAOA, this is done in a closed loop with the quantum hardware, which is time and resource consuming. Best practices on how to carefully choose transpilation and error suppression strategies for a QAOA benchmark execution can be found on GitHub[47]. In our case, we choose to optimize β,γ and λ on classical hardware by evaluating $\langle H\rangle$, which is efficiently possible for a depth-one ansatz[48]. Optimizing λ is a hard task[49]. Here, we chose to strike a balance between the impact of the actual objective part of the Hamiltonian and the constraint part of the Hamiltonian. This can be formulated, for example, as the optimization problem

$$\max_\lambda \min\left(\langle H_{\text{obj}}\rangle^\star,\lambda\langle H_{\text{const}}\rangle^\star\right).$$

More specifically, we are aiming to find the largest λ that still gives us the correct solution for the objective function underlying our problem. Here the expectation values $\langle\cdot\rangle^\star$ are taken over $|\psi(\beta^\star,\gamma^\star)\rangle$ with classically evaluated optimal parameters $\beta^\star$ and $\gamma^\star$ that maximize $\langle H\rangle$ at a fixed λ. Notably, $\beta^\star$ and $\gamma^\star$ are found from the inner optimization

$$\max_{\beta,\gamma}\langle\psi(\beta,\gamma)|H_{\text{obj}}+\lambda H_{\text{const}}|\psi(\beta,\gamma)\rangle=\max_{\beta,\gamma}E(\beta,\gamma;\lambda).$$

It should be noted that the operator in the expectation value is now the full cost Hamiltonian H and not the potentially simplified H'. β and γ are optimized with COBYLA, provided by ScipPy. Crucially, these QAOA parameters depend on λ. Therefore, the Lagrange multiplier optimization is expressed as

$$\max_\lambda \min\left(\langle H_{\text{obj}}\rangle^\star,\langle H\rangle^\star-\langle H_{\text{obj}}\rangle^\star\right)=\max_\lambda F(\lambda).$$

We optimize λ by performing a linear scan over the intervals [0, 1] and [0, 0.2] for the 17- and 52-qubit instances, respectively, followed by a refinement using COBYLA from SciPy[50]. The resulting optimized λ values are 0.528 and 0.119 for the 17- and 52-qubit instances, respectively.

Table 2 | Independent Set: runtimes

n	Total runtime	Preprocessing (s) (t_{opt}; t_{circ})	QPU runtime (s)	Post-processing (s)	Best obj. val.
17	77	36 (22; 14)	41	0	4 (optimal)
52	227	187 (175; 12)	38	2	13 (optimal)

Summary of benchmarking results for mamila kangaroo and aves sparrow stable set instances. The process was executed once. The overall runtime, reported in seconds, is broken down into (1) the preprocessing time, which includes the time t_{opt} to obtain $\beta^\star$, $\gamma^\star$, $\lambda^\star$ and t_{circ} to create the quantum circuits, (2) the QPU runtime, which includes the payload quantum circuit preparation and execution (queuing time is not included), and (3) the time taken to post-process the samples.

To make the lowest-energy state of H correspond to the optimal solution of the maximum independent set problem, $\lambda > 1$ is required. However, we can only afford large values of lambda if we run deep QAOAs with $p > 1$ since they have a better resolution than QAOAs with $p = 1$. Since hardware noise restricts us to shallow circuits, we use $p = 1$ and work with small values of λ. Once the optimal parameters are found, we create a quantum circuit corresponding to $|\psi[\beta^\star(\lambda^\star), \gamma^\star(\lambda^\star); \lambda^\star]\rangle$ and sample 1,024 candidate solutions on ibm_fez, a 156-qubit Heron R2 superconducting qubit processor[51]. These circuits are executed in session mode as this allows us to report the quantum processing unit (QPU) time as the payload quantum circuit preparation and execution time—as discussed in 'Computational resources'. Finally, the samples are post-processed by the greedy classical method described in Supplementary Information—Problem classes—Maximum independent set.

This benchmark was executed using NumPy v.1.23.5, Qiskit v.1.2.2 and SciPy v.1.11.3, all running on Python v.3.11.5. We benchmark this approach on stable set instances with 17 and 52 variables referred to as mamila kangaroo and aves sparrow, which are given in refs. 5. All constraints of the 17-variable instance are implemented on hardware, that is, $H_{const}' = H_{const}$. This requires a total of 308 CNOT (controlled NOT) gates. For the 52-variable instance, we only implement 16.7% of the constraints by allowing three layers of SWAP gates[52]. We therefore only implement the constraints that can be built up on a line of qubits on which we apply three layers of SWAP gates as described in ref. 52. The resulting circuit has a total of 207 CNOT gates. This keeps the fidelity of the circuit, approximated by $(1 - E_{CZ})^{n_{CZ}}$, above 50%. Here E_{CZ} and n_{CZ} are the median error per two-qubit controlled Z (CZ) gate and the number of two-qubit CZ gates, respectively. With the 52-variable instance, we do not sample any feasible solutions. However, after the greedy classical post-processing, we obtained the optimal sample. The runtime results from the process described above are reported in Table 2. As expected, most of the runtime is dedicated to optimizing β, γ and λ. We observe that 60% and 27% of the post-processed samples for the 17- and 52-decision-variable problems, respectively, are optimal (Fig. 2). Table 3 summarizes the quantum benchmark for the 52-variable instance.

Discussion

While complexity theory suggests that the speed-up for finding provably optimal solutions to NP-hard problems with quantum algorithms can generally be at most quadratic[53–56], better quantum approximation algorithms with up to an exponential speed-up[57] or quantum optimization heuristics that outperform classical algorithms on certain instances may still exist. However, practical demonstrations of quantum advantage in optimization are still missing. To track progress towards potential quantum advantage, it is key to identify which problems are best suited to explore possible avenues. The selection of problem classes for this library is primarily based on their difficulty for classical methods, and, to some extent, on their prominence in existing quantum approaches. Furthermore, tracking the progress in algorithmic methodology and hardware capabilities is facilitated through fair and rigorous submission guidelines—see Table 4. The metrics were chosen to enable a systematic comparison of solutions and algorithms. Our aim is to include submitted solutions on a rolling basis, and problem instances or adaptations of existing ones may be added as the state of the art evolves. More specifically, we plan on treating the library[5] as a living repository by keeping it up to date with novel findings and methodologies. Hence, we intend to supplement the main library with additional problem classes or update existing ones as needed. If new insights reveal that certain classes are more appropriate, or if new methods render existing classes easier to solve, we will revise the library accordingly. Furthermore, future submissions may show the need to update or develop the respective metrics. In this case, we will improve the respective guidelines. We invite all researchers interested in advancing (quantum) optimization to investigate the provided problem instances and submit solutions using algorithms and hardware of their choice. To accelerate the overall research, scientists are encouraged to also report results for approaches that did not work well. Additionally, we emphasize the importance of improving classical methodologies, among other reasons, because quantum advantage can only be claimed if the best known classical methods are outperformed.

Methods

This section presents the considered problem types, possibilities to model them and metrics to measure the performance. See 'Results' for a detailed explanation of the maximum independent set class. The description of the specific instances and the baseline results computed with classical optimization solvers can be found in Supplementary Information—Problem classes. A detailed explanation of how the classical baselines are computed is further given in Supplementary Information—Classical benchmark environment. We do not claim that our baseline results are the best solutions one can find with a classical computer, but they should provide an indication of what is possible using general solvers without extensive effort. Moreover, the provided MIP and QUBO formulations were not subject to extensive optimization efforts but are intended to serve as illustrations and were run on a compute cluster using mostly default parameters. Of course, all instances, used models and solutions are also made accessible in the repository[5].

Problem types

We consider ten classes of combinatorial optimization problems. Reference 58 describes combinatorial optimization as follows: "Combinatorial Optimization searches for an optimum object in a finite collection of objects. Typically, the collection has a concise representation, while the number of objects is huge—more precisely, it grows exponentially in the size of the representation. So scanning all objects one by one and selecting the best one is not an option". There are other classes of optimization problems for which quantum solvers have been proposed, including several classes of polynomial-time solvable convex optimization problems; see ref. 1 for an overview. We focus on combinatorial optimization for the following two reasons. First, quantum algorithms for convex optimization usually require fault-tolerant quantum computers. Hence, they cannot be tested on existing quantum devices. For combinatorial optimization, on the other hand, multiple (heuristic) approaches are known that can be implemented and tested already today. Second, classical computers excel at solving convex optimization problems even at massive scales, while combinatorial optimization problems can be intractable for classical algorithms even at a relatively small number of variables. This increases the possibility that near-term quantum computers could outperform classical ones in certain cases.

More specifically, we consider combinatorial optimization problems with $n \in \mathbb{N}$ decision variables of the form

$$x^* = \operatorname*{argmin}_{x \in X} f(x),$$

with feasible space $X = \{x \in \mathbb{Z}^n | g(x) \leq b, l \leq x \leq u\}$, where $g : \mathbb{Z}^n \to \mathbb{Z}$, $b \in \mathbb{Z}, l, u \in \mathbb{Z}^n$, and objective function $f : X \to \mathbb{Z}$. It should be noted that

https://doi.org/10.1038/s43588-026-00991-1

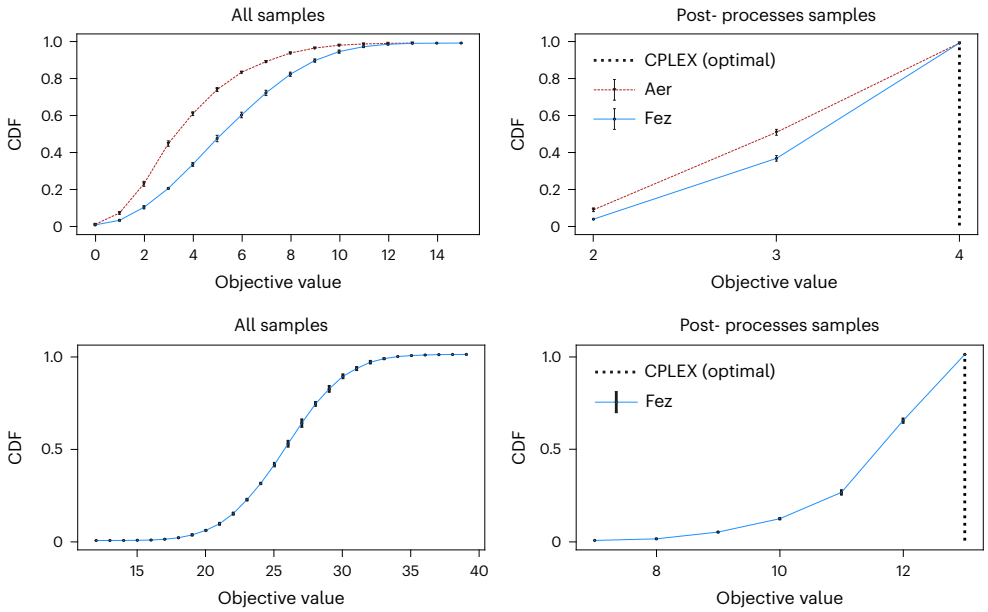

Fig. 2 | Independent Set: cumulative distribution function (CDF) objective values. Cumulative distribution of the objective value corresponding to the samples for the mamila kangaroo (top) and the aves sparrow (bottom) problem instances before classical post-processing (left) and after post-processing of the quantum samples (right). The mamila kangaroo problem is small enough to simulate the QAOA on a classical processor (dashed line). The samples from the ibm_fez quantum processor are shown as the solid blue line.

the coefficients are (if necessary) rescaled and rounded to integers to mitigate potential numerical accuracy issues and to simplify the solution. We distinguish between feasibility problems and optimization problems. For feasibility problems, we can assume that f is constant, and we are interested in finding any solution $x^* \in X$ that satisfies all given constraints, or in showing that $X = \emptyset$. For optimization problems, we have a non-constant objective function to be minimized or maximized, in addition to potential constraints to be satisfied. If an algorithm returns a solution $\hat{x} \in X$, we call the solution feasible. If in addition $f(\hat{x}) = \min_{x \in X} f(x)$, we call the solution optimal. Multiple optimal solutions may exist.

From the theoretical side, the two problem types are closely related, as we can solve optimization problems via a sequence of feasibility problems by adding a constraint for a certain objective value k, that is, $X_k = \{x \in X | f(x) \leq k\}$, and asking whether $X_k = \emptyset$. The value of k can then be iteratively adjusted—for example, using binary search—until the optimal value is found. We can also make the problem unconstrained: for instance, by removing the constraint and instead minimizing $\tilde{f}(x) = f(x) + M \max\{0, g(x) - b\}$, $X = \{x \in \mathbb{Z}^n | l \leq x \leq u\}$ for a suitably large M (see Modeling for a more detailed discussion).

In practice, a good solution without proof of optimality or provable bound on the approximation ratio is often sufficient. If a formal guarantee of the solution quality is not required, finding a feasible solution with a good objective value is often achievable with classical heuristics, although challenging problem instances exist. We restrict our benchmarking study to computing good solutions without requiring a proof of optimality: first, because we want to assess practical performance, and second, because most known quantum optimization algorithms are heuristics and do not provide any a priori or a posteriori performance guarantees.

Modeling

Mathematical programming has become a fundamental tool in many disciplines thanks to the existence of high-performance solvers that can find good—often even optimal—solutions for large classes of problems. Mathematical programming provides a way to classify models according to their structure and match model families with their respective specialized algorithms. The invention of more advanced solvers led to a shift in how problems are modeled and solved. As an example, the impactful success of solvers for MIP resulted in many applications that were reformulated into linear formulations to benefit from highly performant implementations. Modern MIP solvers can solve—often to proven optimality—mixed integer linear and quadratic programs, as well as their pure integer and binary variants: integer linear programs (ILPs), integer quadratic programs, binary linear programs and binary quadratic programs, respectively.

The choice of problem formulation, even in the same model class, can have a tremendous impact on the solvability of a problem[59]. While the global optimum is typically the same for different problem formulations, one might be better suited for a particular problem instance or algorithm than another. The development of novel hardware and algorithmic devices to solve QUBO problems has led to a renewed interest in formulations of problems as QUBOs[60]. While all ILPs or integer quadratic programs can be arbitrarily approximated by some QUBO, naive reformulations will often contain a large number of densely connected variables and will require outstanding precision for the coefficients to accurately represent the original structure[49]. For example, ILPs can explicitly represent different constraints or variables that can be difficult to encode into a QUBO, since many constraints can only be incorporated implicitly into QUBOs via penalty terms that vanish for feasible solutions. In addition, to realize a QUBO representation, integer variables must be decomposed into binary ones—a useful library to handle this conversion can be found in ref. 61. Direct translation from a general ILP to a QUBO requires making decisions on hyperparameters that define the encoding and penalty functions and can impact the quality of a solution depending on the considered algorithm. Furthermore, comparing the results received from multiple formulations of a problem requires careful interpretation. The objective values of two QUBOs representing the same problem using different penalty factors cannot be directly compared. Therefore, solutions must be mapped back to the original problem to understand and compare their performance. Higher-level modeling languages exist that provide additional structure to accommodate a problem's original semantics more naturally, and multiple tools can automate reformulating from a high-level representation into, for instance, an ILP or QUBO. Thus, the fact that a

https://doi.org/10.1038/s43588-026-00991-1

Table 3 | Independent Set: quantum submission metrics

Problem identifier	aves-sparrow-social.gph
Submitter	**Daniel J. Egger (IBM Research Europe—Zurich)**
Date	**15 January 2025**
Reference	164
Best objective value	13
Optimality bound	N/A
Modeling approach	QUBO
No. of decision variables	52
No. of binary variables	52
No. of integer variables	0
No. of continuous variables	0
No. of non-zero coefficients	506 (number of edges in graph + number of nodes)
Coefficient type	Continuous
Coefficient range	[0, 2]
Workflow	(1) The parameters β and γ of the depth-one QAOA and the
	7D1 Lagrange multipliers are optimized classically.
	(2) Samples are drawn from the QPU.
	(3) Samples from the QPU are classically post-processed.
Algorithm type	Stochastic
No. of runs	5
No. of feasible runs	5
No. of successful runs	5
Success threshold	0
Hardware specifications	CPU, Intel Core i9-10885H (Levono P1); QPU, ibm_fez
Total runtime	252 s
CPU runtime	192 s
GPU runtime	N/A
QPU runtime	60 s
Other HW runtime	N/A

Benchmark submission, for instance aves-sparrow-social from the Independent Set benchmarking problem collection provided in the QOBLIB[5] repository. GPU, graphics processing unit; HW, hardware.

problem can be cast into a certain form does not necessarily mean that, in practice, it is a good idea to solve it in that form. Given the inherent formulation differences, we focus on model-independent benchmarking such that we may find the best formulation and corresponding algorithm for a given problem instance. Another type of reformulation allows mapping of higher-order unconstrained binary optimizations, which may either be solved directly[62] or converted to QUBOs[63]. To this extent, higher-order binary terms are broken into multiple quadratic terms by introducing additional binary variables. Depending on the number and degree of the higher-order terms, this can increase the number of binary variables, potentially leading to impractical problem formulations. Some quantum optimization algorithms can handle higher-order terms natively, giving them a potential advantage over those algorithms (quantum and classical) that cannot. Notably, many quantum optimization algorithms are designed for QUBOs. A common approach is, thus, to start with a higher-level problem formulation and reformulate it as QUBO. Tutorials on QUBO reformulations can be found, for example, in refs. 64,65.

Table 4 | Submission metrics

Problem	Identifier of the considered problem instance
Submitter	**Name(s) of the submitter(s) and affiliation(s)**
Date	**Date of submission**
Reference	Reference to a paper/repository with more details
Best objective value	The best objective value found by the algorithm across all repetitions
Optimality bound	Lower bound (minimization) or upper bound (maximization) for the optimal objective value, if supported; otherwise, set to N/A
Modeling approach	Describe how the considered problem instance is modeled
No. of decision variables	Total number of decision variables
No. of binary variables	Number of binary decision variables
No. of integer variables	Number of integer decision variables
No. of continuous variables	Number of continuous decision variables
No. of non-zero coefficients	Number of non-zero coefficients in objective function and constraints
Coefficient type	Type of coefficients, such as integer, binary, continuous
Coefficient range	Range of non-zero coefficients, that is, min./max. values
Workflow	Description of the optimization workflow: preprocessing, presolvers, machine learning training, optimization algorithms, post-processing and so on
Algorithm type	Indicate whether the algorithm is deterministic or stochastic
No. of runs	The number of times the experiment has been repeated
No. of feasible runs	The number of times a run found a feasible solution
No. of successful runs	Number of runs that found a feasible solution with objective value $\leq(1+\epsilon)^* f_{\min}$ (minimization) or $\geq(1-\epsilon)^* f_{\max}$ (maximization), where $f_{\min/\max}$ is the best solution found by the algorithm
Success threshold	The threshold ϵ to define a successful run
Hardware specifications	Specifications of hardware used to run the workflow
Total runtime (wall clock)	Total runtime to run the complete workflow
CPU runtime	CPU runtime to run the workflow
GPU runtime	GPU runtime to run the workflow
QPU runtime	QPU runtime to run the workflow
Other HW runtime	Runtime on other hardware to run the workflow
Remarks	Any additional remarks

This table summarizes the metrics that should be reported when submitting benchmark results. All runtimes should be reported as average if multiple algorithm runs were executed.

Metrics

The different possible goals of running a benchmark, as described in the Introduction, require different metrics to be considered. To ensure that benchmarking results allow for a fair comparison, the reported metrics must be clearly defined. In the following, we will focus on solution quality and required computational resources and discuss the corresponding metrics. Examples of how to report benchmarking results for selected problem instances and quantum algorithms are available in refs. 5.

Solution quality. The metrics for evaluating solution quality differ between feasibility and optimization problems. A feasibility problem

is completely solved as soon as a feasible solution is found. However, proving that no such solution exists is usually much more challenging, and impossible for most heuristics. For optimization problems, the key metric to be reported is the best objective value corresponding to a feasible solution found by an algorithm. If the algorithm also provides a lower bound (for minimization) or an upper bound (for maximization) for the optimal objective value, this bound can be provided as well. To solve an optimization problem to proven optimality, an algorithm must first find an optimal $x^* \in X$ and then prove that no better $x \in X$ exists with $f(x) < f(x^*)$—for example, by providing a tight bound on the objective value. Usually, the second step is the more difficult one.

The above respective algorithms may run deterministically or, as many classical and most quantum algorithms do, stochastically. In the latter case, additional details should be reported. Stochastic algorithms should be executed multiple times to ensure that the reported results describe the typical behavior of the algorithm. Then, for both feasibility and optimization problems, the number of repetitions yielding feasible solutions should be reported. For optimization problems, additionally, the best objective value achieved by a feasible solution found across all repetitions should be stated, as well as the number of successful repetitions. Given a number of repetitions of a stochastic algorithm and a given tolerance $\epsilon \geq 0$, we consider a single run successful if it finds a feasible solution with an objective value that is ϵ-close to the best found feasible solution across all repetitions. More precisely, consider the best found solution over all repetitions, denoted by x' with corresponding objective value $f(x')$, which may differ from the global optimum $f(x^*)$. Then, a solution x corresponding to a successful run must satisfy $f(x) \leq (1+\epsilon)f(x')$ (minimization) or $f(x) \geq (1-\epsilon)f(x')$ (maximization)—assuming non-negative objective values.

Notably, we may also want to track metrics that present combined information about solution quality and the time it takes to arrive at a particular quality. One example thereof is the primal integral[66], which tracks the primal bound towards the optimal (or best known) solution throughout the iterations executed by a solver. This approach has been found to be suitable for comparing the performance of solvers that do not necessarily return optimal solutions. This includes machine learning-based optimization solvers[67,68]—trained on a large number of known instances—which can, in certain cases, predict good solutions quickly.

Computational resources. To compare different methods and algorithms, it is crucial to transparently report the type of computational resource used and the duration for which they were utilized. Thus, software versions and hardware specifications, such as the type of processing unit used and the amount of memory available, must be reported, as discussed in more detail below.

First, the total runtime (s), that is, the total wall-clock time for running the considered algorithm using the reported hardware, excluding idling times, for example, queuing for execution on a (quantum) computer while the remaining processing units are waiting. For stochastic algorithms that are repeated multiple times, the average total runtime over all repetitions should be reported. If the algorithm is parallelized and uses multiple processing units simultaneously, this should be noted in the hardware description but not translated into a single-core runtime. The definition of the total runtime, excluding idling times, can be easily measured and represents the runtime most relevant for practical applications. In certain cases, it may also be useful to report the time-to-solution, that is, the time from the algorithm's start until the returned solution was found.

Second, to better understand how different hardware is utilized, reported runtimes should be split into the execution time on CPUs, GPUs, quantum computers and alternative hardware, such as, for instance, field programmable gate arrays or Ising machines. How to accurately define, measure and report runtime on a quantum computer is discussed in 'Quantum computing runtimes'.

There are many additional details of interest, such as the intermediate problem sizes after presolving or the time and resources spent in different stages of the workflow: presolving, preprocessing (circuit preparation and so on), training of machine learning-based solvers, post-processing and so on. While it is difficult to capture all of these details in a submission table, we strongly encourage benchmark submitters to provide such details in an accompanying reference, such as a paper or repository.

Quantum computing runtimes. In the following, we define quantum computing runtime and how to measure it. A quantum computer consists of multiple stages, including a runtime environment, such as Qiskit Runtime, the control electronics and the actual quantum chip. The payload sent to a quantum computer needs to pass these stages, and even though some steps are performed using classical computing, such as, for example, compiling a quantum circuit to microwave pulses, we attribute them to the runtime on the quantum computer. Thus, we define the runtime of a single run of a quantum (sub-) routine as the total time it takes to receive the payload, prepare it, run the circuit on the QPU and extract and return samples from the system by measuring the qubits. We would like to highlight that the number of samples drawn from a quantum circuit corresponds to a hyperparameter that has to be chosen as part of the considered quantum algorithm. Furthermore, we assume that circuits are provided using a topology and gate set that is natively supported by the quantum computer. Thus, while we include the time it takes to prepare such a circuit for execution, for example, to microwave pulses, we explicitly exclude the time it takes to map a given circuit to the instruction set supported by a particular quantum computer. This additional transpilation time should be considered during classical preprocessing. Similarly, we attribute time spent on error mitigation to the quantum computer if it happens as part of the runtime environment, and to the classical processing units if it is handled via post-processing.

Finally, we describe how to measure the quantum computing runtime in the most reliable way on the example of the IBM Quantum Platform[51,69]. When using an IBM Quantum computer, quantum circuits should be run using the Qiskit Runtime session mode[70] instead of batch or job mode. Once a session has been started, it blocks the whole quantum computer for a user, in contrast to batch and single-job modes, which may share resources—such as circuit preparation pipelines—across different users and thus may not provide accurate runtime estimates. After a job has been executed, the results returned from the IBM Quantum Platform[51] contain the session start and end times, as well as the times individual jobs were submitted, started and completed. In addition, the results report actual usage, which refers to the time that was actually spent on the QPU, excluding classical pre- and post-processing inside the quantum computer. Due to the potential parallelization of circuit preparation pipelines before executing them on the QPU, we cannot just add the runtimes of the individual jobs. Instead, we take the total time the session is active, that is, the difference between start and end times excluding initial queuing time, and further subtract all times where the session was active but idling, that is, while no jobs were submitted and we were not waiting for previously submitted jobs to be completed. This is what we define as the quantum computer runtime, while the idling time in between jobs should be attributed to, for example, the classical runtime.

Problem classes

Combinatorial optimization problem instances can be loosely classified into three categories, which are not always mutually exclusive. Real-world instances arise from real-world problems, and are therefore crucial for demonstrating a practical quantum advantage. Since there is already a demand for a solution, such instances have often been modeled (and possibly simplified) such that they can be tackled by existing solvers. The availability of real-world instances to the scientific

community is often limited, even more so for unsolved problems as discussed above. Random instances are based on randomly generated data, which may be motivated by real-world problems or completely artificial. Depending on the generation method, such instances often do not capture the structural peculiarities of a real-world setting, and may be easy or hard to solve. Crafted instances are most likely the best way to construct artificial examples that are hard to solve. However, there is a risk that the instances are of no practical relevance, as they often represent borderline pathological cases. A quantum advantage for random or crafted instances is interesting, as it may help to identify real-world problems with similar structures. The problem instances presented here were mostly crafted to fit a range of criteria we identified as crucial for a useful optimization benchmarking framework, while they are still mostly related to real-world problems.

Problem instances can be further classified as primal difficult, dual difficult or both. Primal difficulty refers to instances where it is difficult to find an optimal solution; dual difficulty refers to instances where it is difficult to prove a solution is optimal. An astonishing number of instances are primal easy but dual hard. Before solving an instance, it is a priori unclear which is the case. One may, however, gain intuition about the difficulty of a problem instance from solutions of other instances of the same problem class, since instances within the same class often behave similarly. Since most currently known quantum optimization algorithms are heuristics, we consider the chances for demonstrating a near-term quantum advantage higher for primal difficult problems and do not yet expect a benefit for proving optimality, although that might be possible later. Hence, we mainly present problems that are considered primal difficult. Further, if it is easy to find a very good or even optimal solution, the problem is not difficult from a practical point of view, and thus, there is less potential for demonstrating practical quantum advantage. Furthermore, we would like to note that we describe an informed approach to construct warm-up examples from the provided problem instances in Supplementary Information—Instance simplification, which can help to build up expertise and start tracking today's capabilities.

This section provides the background and problem description of the selected ten problem classes. The specific instances and respective classical baselines are given in Supplementary Information—Problem classes. The number of decision variables for which MIP or QUBO formulations of the selected instances become difficult ranges from less than 100 to 45,000. Depending on the problem, the ranges of the constraint coefficients lie between 1 and ~10^7, assuming that the coefficients are scaled to integers, as we do throughout the Resource. This can pose another challenge for quantum computers as they need to be able to represent the problem data in the necessary resolution.

Many of the problem instances are already suitable for testing with current quantum computers. Some can be addressed using methods such as QAOA, which apply a one-to-one variable-to-qubit encoding. Others require more compact approaches, such as Pauli Correlation Encoding[71], which can represent more than one variable per qubit, or decomposition techniques that split large problems into smaller subproblems solvable by quantum computers.

Market Split. *Background*. The Market Split instances are multidimensional subset-sum problems, which were explicitly designed to be hard to solve using classical methods. Since their introduction in ref. 72 in 1999, the progress in solving them has been moderate. Subset sum is a classic NP-hard problem[73]. The time complexity for enumeration is $O(2^{n/2}(n/4))$ (ref. 74).

Why is it interesting?. Market Split problems are a type of multidimensional subset-sum problem. This problem class contains various instances for which it is challenging to find feasible solutions with classical digital systems—even for small system sizes, with little more than 100 variables. Applications of the Market Split problem could be interesting in various industries, such as the energy sector, for example, to help derive optimal pricing strategies subject to customer acquisition, retention or competition constraints, and regional regulatory policies.

Problem statement. Given a matrix $A \in \mathbb{N}^{m\times n}$ and a right-hand side $b \in \mathbb{N}^m$, for $m, n \in \mathbb{N}$, we want to find $x \in \{0,1\}^n$ such that

$$Ax = b. \tag{1}$$

This is a multidimensional subset-sum problem. We can transform equation (1) trivially into unrestricted optimization problems:

$$\min_{x\in\{0,1\}^n} (b - Ax)^2 \text{ or } \min_{x\in\{0,1\}^n} \|b - Ax\|_\infty,$$

where a feasible solution exists if the global minimum is zero. With careful choices of the coefficients, this problem becomes hard to solve, for example, for numbers $m \in \mathbb{N}_+$, $D \in \mathbb{N}$, set $n = 10(m-1)$, $I = \{1, \ldots, m\}$, $J = \{1, \ldots, n\}$ and $a_{ij} \in \{0, \ldots, D-1\}$, $b_i = [\frac{1}{2}\sum_{j\in J} a_{ij}]$ (ref. 72).

The best results so far have been achieved using forms of lattice enumeration with basis reduction. Reference 75 and A. Wassermann (manuscript in preparation) report solutions to selected instances up to $m = 14$. Traditional branch-and-cut-based MIP solvers often already struggle before size $m \geq 7$. However, they are typically more effective on the lattice reformulation introduced in ref. 76.

Low-autocorrelation binary sequences. *Background*. LABS were initially studied in the context of radar and sonar system design, where achieving minimal autocorrelation sidelobes is crucial for improving range resolution and target discrimination[77]. Over time, these sequences have also proved useful in digital communications, coding theory and spread-spectrum systems, where low-autocorrelation properties can enhance channel efficiency and signal clarity. Unlike many hard optimization problems, each sequence length N in LABS defines a single, unique problem instance. As a result, progress on LABS is easy to measure: pushing beyond known solutions or improving runtime for larger N directly reflects meaningful advances in algorithmic capability.

LABS also appear as ground states of the Bernasconi model in statistical physics, a system involving complex, long-range four-body spin interactions[78]. Despite non-negligible research efforts, finding optimal LABS remains a challenging task. LABS instances can be solved using convex-relaxation-based branch and bound. In this line of research, LABS is formulated as a polynomial optimization problem over 0–1 variables and solved using branch-and-cut techniques. Both off-the-shelf solvers and specialized solvers can be used to deal with the corresponding formulation, but the exact solution of large instances remains difficult; see ref. 79 as well as the benchmark results (with optimality gaps) in MINLPLib[80]. For convex-relaxation-based branch and bound, the empirical performance on LABS instances varies greatly, and it is difficult to estimate. The empirically estimated running time of a tailored classical branch-and-bound method that uses combinatorial bounds[81] is approximately 1.73^N, while the best-performing heuristic discussed in the literature for this problem (memetic tabu search[82]) has an estimated running time of approximately 1.34^N; see ref. 83 and references therein for a discussion of these running times and ref. 84 for recent results. The work presented in ref. 83 also shows an empirical study, extrapolating from results on smaller instances, estimating the complexity of quantum optimization approaches, particularly those based on QAOA combined with amplitude amplification, to be around 1.21^N, considering idealized conditions. Although these figures are estimates for exponential-time algorithms, which are notoriously difficult to estimate correctly on unseen instances, they raise the possibility that quantum approaches could eventually outperform the fastest known classical algorithms.

Why is it interesting?. Established classical algorithms struggle to find (good) solutions to LABS instances even at relatively small sizes. Moreover, the binary (spin-like) structure of LABS aligns naturally with Ising-type models, making it a suitable test case for quantum optimization techniques. Demonstrating improvements in scalability or solution quality on LABS would represent an important step in addressing a known hard problem.

Problem statement. Given a sequence $S = (s_1, \ldots, s_n)$ of length n with binary variables $s_j \in \{-1, +1\}$, the minimum energy autocorrelations of the sequence $j \in \{0, \ldots, n-1\}$ corresponds to

$$\min_{s\in\{-1,+1\}^n} \sum_{j=1}^{n-1} \left(\sum_{i=1}^{n-j} s_i s_{i+j} \right)^2 .$$

As LABS is an unconstrained problem, any sequence S is a feasible solution.

Minimum Birkhoff decomposition. *Background*. The minimum Birkhoff decomposition problem is a special case of the Birkhoff decomposition problem[85]. In 1946, Birkhoff showed that any $n \times n$ doubly stochastic matrix can be written as a convex combination of permutation matrices[86]. The method of proof used by Birkhoff inspired a class of algorithms for decomposing doubly stochastic matrices known as the Birkhoff algorithm (ref. 87, p. 192), which can decompose a doubly stochastic matrix with at most $(n-1)^2 + 1$ permutation matrices[88]. It is possible to obtain tighter upper bounds on the number of permutations required depending on the number of zeros in the doubly stochastic matrix; however, the lower bound, that is, the minimum number of permutation matrices required, is generally not known. In ref. 85, Dufossé and Uçar showed that the problem of finding the minimum Birkhoff decomposition is NP-hard despite the existence of algorithms that can obtain approximate decomposition where the error decreases exponentially with the number of permutations in the decomposition[89].

Decomposing doubly stochastic matrices has applications in assignment problems, especially in networking[90–92]. In these problems, a permutation matrix corresponds to a matching in a bipartite graph, which represents a schedule or system configuration used to exchange some resource (for example, data packets, energy). The weight associated with a permutation matrix captures the time the system will spend in such a configuration. Sparse decompositions are important to reduce the time a system spends switching between configurations, which is non-negligible in practice. Studying this problem can also be of interest for more general graph scheduling problems where the underlying graph is not just bipartite—for example, in peer-to-peer networks[93].

The algorithms for solving the (minimum) Birkhoff decomposition methods can be divided into three classes: branch-and-bound algorithms for solving a mixed-integer nonlinear program[94], Frank–Wolfe (FW) algorithms[95] and variants of the Birkhoff algorithms[92,96]. Each approach has different characteristics. Branch-and-bound algorithms are often the slowest ones but can obtain certifiable optimal solutions when the doubly stochastic matrices are not too large or too dense. In contrast, FW algorithms are very fast and can obtain coarse approximations with just a few iterations. However, they are often unable to obtain exact decompositions even when the number of permutations used exceeds $(n-1)^2 + 1$. Birkhoff-type algorithms take an approach that can be considered to be between FW and branch-and-bound methods: they can obtain exact decompositions with at most $(n-1)^2 + 1$ permutations, and their speed is comparable to that of FW algorithms. Unlike branch and bound, Birkhoff-type algorithms do not guarantee that the solution obtained is optimal.

Why is it interesting?. The problem of finding the minimum Birkhoff decomposition is NP-hard[85], and it is hard to solve in practice, even for small instances. Also, it is easy to craft problem instances and obtain non-trivial upper bounds on the optimal cost, which is generally challenging in NP-hard problems. Another interesting aspect is that the problem of decomposing doubly stochastic matrices is well studied, and many mathematical formulations and algorithms exist, including MIP and convex optimization. We also note that doubly stochastic matrices have a natural representation in terms of unitaries[97], which are the backbone of quantum computation.

Problem statement. Let D be an $n \times n$ doubly stochastic matrix and P_i the ith $n \times n$ permutation matrix, $i \in \{1, \ldots, n!\}$. Recall that a matrix is doubly stochastic if its entries are non-negative and the rows and columns sum to one. Similarly, a permutation matrix is a doubly stochastic matrix with binary entries.

For a given doubly stochastic matrix D, the minimum Birkhoff decomposition problem is

$$\min_{c_i\in[0,s]} \sum_{i=1}^{n!} |c_i|^0 \quad \text{subject to} \quad \begin{array}{l} sD = \sum_{i=1}^{n!} c_i P_i \\ \sum_{i=1}^{n!} c_i = s \end{array},$$

where we define $0^0 = 0$, that is, $|c_i|^0$ counts the number of non-zero entries and $s > 0$ is a scalar, often equal to one. The goal is to find a subset of permutation matrices such that D is in its convex hull and that this subset contains as few permutation matrices as possible. The following is an example of a decomposition of a 3×3 matrix:

$$\begin{bmatrix} 0.2 & 0.3 & 0.5 \\ 0.6 & 0.2 & 0.2 \\ 0.2 & 0.5 & 0.3 \end{bmatrix} = 0.2 \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} + 0.2 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} + 0.1 \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$+0.5 \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} + 0.0 \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} + 0.0 \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Note that $\sum_{i=1}^{n!} |c_i|^0 = 4$ in the example above since the decomposition only uses four out of the $3! = 6$ possible permutation matrices.

Steiner tree packing. *Background*. The Steiner tree packing problem (STPP) describes the problem of placing n individual Steiner trees node-disjointly into a graph. For $n = 1$ this is the Steiner tree problem, that is, the combinatorial variant of the much older Euclidean Steiner problem. The Euclidean Steiner problem asks for the minimal tree that connects a given set of points in the plane: the special case with three points was discussed by Fermat before 1640, whereas the general version may have originated from Gauss[98]. The Steiner tree problem replaces points with vertices in a graph with given edges and weights, and we are asked to find a minimum cost tree that connects a given set of terminals[99]. There is abundant literature on the Steiner tree problem, which is already NP-hard.

The STPP provides a natural mathematical optimization formulation for very large-scale integration design[100]. Therefore, its efficient solution has tremendous practical interest. Early work on the STPP included polyhedral studies[101], separation routines for valid inequalities[102] and approximation algorithms[103].

Why is it interesting?. The minimization problem underlying the STPP comes with a linear objective function; the density is relatively sparse, the coefficients are all ones and the decision variables are binary. Computing optimal solutions, especially for dense routing areas, proved to

be very hard, and little progress—apart from improvements in hardware and general solvers—has been achieved in the last two decades. This problem is also notoriously difficult for heuristics since, in a dense solution, one wrong decision can render the problem infeasible. In practice, this problem class can be used to model network design problems while considering cost minimization and reliability constraints.

Problem statement. For $n \in \mathbb{N}_+$, we write $[n] = \{1, \ldots, n\}$. In the STPP, we are given an undirected simple graph $G = (V, E)$, and $n \in \mathbb{N}_+$ disjunct terminal sets $T_1, \ldots, T_n$ with $T_i \subseteq V$ for all $i \in [n]$ and $T_i \cap T_j = \emptyset$ for $i, j \in [n]$, $i \neq j$. The goal is to find an edge set $S^* \subseteq E$ of minimal cardinality such that $(V(S^*), S^*)$ is a forest, where $\bigcup_{i \in [n]} T_i \subseteq V(S^*)$ and no vertex from T_i is connected to any vertex of T_j for all $i \neq j$. The general STPP with $n > 1$, as defined above, requires the inclusion of several disjoint Steiner trees into the same graph. A solution is a forest, and each terminal set T_i is connected by a single tree in the forest.

Sports tournament scheduling. *Background*. The design of algorithms to construct sports timetables goes back to at least the 1950s[104]. Particularly well researched are so-called single round-robin sports timetables where each team plays every other team once. Even though a single round-robin timetable can be constructed in time polynomial in the number of teams, the problem becomes NP-hard as soon as some elementary constraints are added. One of the prime difficulties in sports timetabling is the vast size of the solution space: every single round-robin timetable corresponds to an (oriented) 1-factorization of K_n and the other way around, where K_n denotes the complete graph on n nodes. Considering just 14 teams and even ignoring the order of rounds, the permutation of team names and the home–away status of games, more than 1,132,835,421,602,062,347 unique, corresponding timetables already exist[105].

Numerous IP-based methods have previously been proposed to construct sports timetables. See, for example, ref. 106 for a discussion on IP formulations, ref. 107 for a popular decomposition approach known as first-break-then-schedule and ref. 108 for an application of Benders' decomposition.

At the same time, several QUBO-inspired algorithms have been proposed for a related sports tournament problem known as the break-minimization problem. In this problem, the objective is to minimize the overall number of breaks (that is, consecutive games at home or away), given that the pairing of opponents is fixed in each round.

Why is it interesting?. As soon as any side constraints are considered, the complex combinatorial structure makes it very difficult for established heuristic methods to find feasible solutions. For several medium-sized instances, no known (and investigated) method was able to find any feasible solution[109,110]—despite the fact that the generation process ensures the existence of feasible solutions. Furthermore, the problem is well aligned with real-world applications, which typically 'only' differ in terms of a few additional side constraints.

Problem statement. Given a set of teams $T = \{1, \ldots, n\}$ and time slots $S = \{1, \ldots, 2(n-1)\}$, n even, a double round robin corresponds to a perfect matching of teams in each time slot such that over all time slots each team plays precisely once at home against every other team. We call a double round robin 'phased' if each team meets every other team exactly once during the first and last $n-1$ time slots.

The feasibility version of the ITC2021 sports scheduling problem considered in this benchmark is to find a phased double round robin that additionally respects the following hard constraints.

- Capacity constraints. Regulate when teams can play home or away.
  - CA1. Team $i \in T$ plays at least or no more than k home games in time slots $P \subseteq S$.
  - CA2. Same as CA1 but considering opponents as well.
  - CA3. No more than two consecutive home or two consecutive away games.
  - CA4. Teams in $T_1 \subseteq T$ play no more than k home games against teams in $T_2 \subseteq T$ during time slots in $P \subseteq S$.
- Break constraints. A team has a break if it plays consecutively at home or away.
  - BR1. Team $i \in T$ has no more than k breaks during time slots in $P \subseteq S$.
  - BR2. No more than k breaks in total.
- Game constraints. Forbid some games during specific time slots.
  - GA1. At most k games from $G \subseteq T \times T$ during time slots in $P \subseteq S$.

The original version of the ITC2021 sports scheduling problem also allowed the constraints above to be soft, penalizing violations of the soft constraints while strictly adhering to the hard ones. Additionally, the original version of the problem considered two more soft constraints.

- Fairness and separation constraints. Increase the attractiveness and fairness of the tournament.
  - FA2. At any point in time, the difference in the number of home games played between any two teams does not exceed two.
  - SE1. There are at least ten time slots between each pair of games involving the same teams.

Portfolio optimization. *Background*. The problem of quantitative investment and risk diversification in portfolio optimization spans a broad spectrum of methodologies, each incorporating different simplifications, investor priorities, and trade-offs between risk exposure, returns, computational complexity, robustness and out-of-sample performance. A foundational approach is Markowitz's mean–variance optimization, formulated as a convex quadratic program[111]. This framework has been extended to address real-world constraints[112], estimation errors[113], sensitivity to input variability[114] and Bayesian techniques that integrate expert judgment with statistical models[115]. Computational limitations of the methodologies for practically relevant cases[116] have led to the adoption of, for example, fixed-mix strategies that improve model transparency and stability[117,118], hierarchical risk parity and risk budgeting techniques[119,120] and multiperiod approaches that help to capture intertemporal effects and hedging demands[121]. In fact, portfolio optimization models can be defined with varying complexity[122,123] depending on whether one aims to also jointly model risk, return predictability, and impact costs over time while addressing the exponential growth of time complexity with respect to the number of assets and rebalancing periods[124]. Portfolio optimization can transform into a stochastic dynamic programming problem, which, under realistic conditions that account for, for example, transaction costs or short selling, has been proven to be computationally intractable for portfolios with more than two risky assets[125].

Given its relevance and complexity, portfolio optimization has attracted considerable interest from the quantum computing community. Theoretical research suggests that quantum algorithms may offer speed-ups under specific conditions, which may, however, not hold in real-world instances [126–128]. Another line of research has investigated how we may approach simple portfolio optimization instances with near-term compatible quantum algorithms[129–131]. The simplified version of the problem derived here focuses on the computational complexity in the multiperiod setting, using a binary quadratic formulation of the mean–variance model with linear constraints, and perfect knowledge of expected future returns and covariances. This allows for the creation of compact yet computationally challenging problem instances, as the

problem is NP-hard[132], and the systematic benchmarking of current quantum algorithmic capabilities.

Why is it interesting?. A vast array of industrial and scientific domains are interested in solving problems that aim to find an optimal composition of a portfolio of coupled objects with respective observables that typically share a common probability space. (Financial) portfolio management is one of the hardest variants of this problem. It concerns the risk-sensitive allocation of a time-dependent investment budget for a number of diversified financial assets whose behavior varies with time according to unknown stochastic processes, which are driven by external factors. The hardness of this problem can be explored at different levels of complexity, even at a small scale, and as such allows systematic tracking of algorithmic and hardware progress.

Problem statement. Let $\mathcal{T} := \{t_1, \ldots, t_m\}$ be an ordered set of discrete time points such that $0 \le t_1 < t_2 < \ldots < t_m$, representing a finite investment horizon in the future. Consider an artificial deterministic market that is fully described by a closed system of $n \in \mathbb{Z}^+$ financial assets $\{a_i | i = 1 \ldots, n\}$, where each asset a_i has known expected market prices $\{\bar{p}_{it}\}$ at each time $t \in \mathcal{T}$. Assuming a myopic investor in a mean–variance framework and no access to additional information, the objective is to find a multiperiod trading strategy $\mathcal{S}_1 := \{x_{it}\}$ at t_1, where portfolio allocations $x_{it} \in \{0, 1\}$ represent binary investment decisions of up to C normalized capital units $u \in \mathbb{R}^+$ into assets $\{a_i\}$. We define the price p_{it} as follows: let $e_i := u/\bar{p}_{it_0}$, then $p_{it} = e_i \bar{p}_{it}$, that is, we compute how many assets of type i we can buy for one unit of cash at time t_1. The amount of stocks e_i we can get at time t_1 for one unit of cash u is then used for all later time steps, that is, p_{it} tells us what has become out of one cash unit invested into asset i at time t. The investor seeks to balance returns and risk at terminal time t_m, where the temporal returns are given by $(p_{it} - p_{i,t-1})x_{it}$ and the temporal risk is measured through the risk preference λ-weighted positive semidefinite covariance matrix Σ as $\lambda(px)^{\mathrm{T}}\Sigma xp$, while obeying a set of regularizing trading constraints.

These constraints aim to incorporate more realistic real-world conditions, although here simplified into a quadratic optimization problem.

- Transaction cost. Rebalancing of x over $\mathcal{T}$ incurs costs due to fees, taxation and so on that negatively offset the returns by a factor of transaction cost $\delta \in \mathbb{R}^+$ for each change $p_{it}|x_{it-1} - x_{it}| = p_{it}(x_{it-1} + x_{it} - 2x_{it-1}x_{it})$ and a liquidation cost $\delta p_i x_i$ at terminal time t_m. This is a strong simplification, as cost factors are neither global nor time-invariant scalars, but rather stochastic functions driven by market uncertainties.
- Short selling cost. Given a subset $f \in \mathcal{F} \subseteq \{a_i\}$ of assets permissible for short selling positions over $\mathcal{T}$, a fixed loan rate for short positions $\rho \in \mathbb{R}^+$ is applied to each allocation $\rho p_{ft} x_{ft}$ with $f \in \mathcal{F}$.
- Cash flow limits. Let $u \in \mathbb{R}^+$ denote the unit value of cash, and C the total available units of normalized capital. Each asset is assigned an indicator $\tau_i \in \{-1, +1\}$, representing a short (−1) or long (+1) position. At any time, the total net investment $\sum_i \tau_i x_{it}$ must not exceed the capital limit C, and the total number of assets held at each time step must be bounded by B, which serves as an upper limit on the number of acquired positions.
- Cash interest. Temporarily unused capital earns risk-free interest $vu(C - \sum_i \tau_i x_{it})$ with a risk-free rate $v \in \mathbb{R}^+$.
- Multiple shares. We also allow that each asset i can be selected up to k times. We use a unary encoding where the Hamming weight of k binary variables encodes the number of shares, that is, we essentially consider k copies of the same asset that can be selected. For the small number of shares considered here, this is as efficient as other encodings, but it has the advantage that we can easily model the transaction costs. Thus, we expand the vector of portfolio allocations to a $2kn$-dimensional binary vector $\mathbf{x}$ for the n assets at each time t, where each block of k binary variables represents a single asset type. The inclusion of a factor of 2 accounts for the allowance of short selling, with an equal number of vector elements allocated to represent both long and short positions.

The problem can then be formulated as a binary quadratic program, where the objective is a quadratic function subject to linear constraints, and the decision variables are restricted to be binary:

$$\min_{x \in \{0,1\}^{n \times m}, y \in \{0,1\}^{|\mathcal{C}| \times m}} \sum_{t=1}^{m} \Bigg(\overbrace{\lambda \sum_{i=1}^{n}\sum_{j=1}^{n} \tau_i p_{it} x_{it} \sigma_{ijt} \tau_j p_{jt} x_{jt}}^{\text{risk}} - \underbrace{\sum_{i=1}^{n-1} \tau_i (p_{i,t+1} - p_{it}) x_{it}}_{\text{return}} - \underbrace{\sum_{i=2}^{n} \delta p_{it} (x_{it-1} + x_{it} - 2x_{it-1}x_{it})}_{\text{transaction cost}} - \underbrace{vu \sum_{c \in \mathcal{C}} 2^c y_{ct}}_{\text{cash interest}} + \underbrace{\rho \sum_{f \in \mathcal{F}} p_{ft} x_{ft}}_{\text{short selling cost}} \Bigg) + \underbrace{\delta \sum_{i=1}^{n} (p_{i1} x_{i1} + p_{im} x_{im})}_{t_1 \text{ transaction+liquidation cost}}$$

subject to

$$\sum_{i=1}^{n} \tau_i x_{it} + \sum_{c \in \mathcal{C}} 2^c y_{ct} = C \text{ for all } t \in \mathcal{T} \tag{2}$$

$$\sum_{i=1}^{n} x_{it} \le B \text{ for all } t \in \mathcal{T} \tag{3}$$

where (2) describes the capital limit and (3) limits the number of assets that can be included in the portfolio. Since we restricted the problem to binary variables, we modeled the amount of available cash units as binary variables $y_{ct} \in \{0, 1\}$ with a suitable $c \in \mathcal{C} := \{0, \ldots, \lfloor \log_2(C) \rfloor\}$ such that $\sum_{c \in \mathcal{C}} 2^c \ge C$. It is worth noting that alternative formulations may also be possible.

Network design. *Background*. Network design models have wide applications in telecommunications, transportation planning[133,134] and power networks[135]. One common formulation for this problem is to use an MIP formulation[136]. Given an $n \times n$ matrix T, where each entry t_{ij} represents the traffic to be routed between vertex i and j, and an integer $p > 0$, construct a simple directed graph $D = (V, A)$ with vertex set $V = \{1, \ldots, n\}$ and a subset of selected edges $S^* \subseteq A$, where each vertex has in-degree and out-degree equal to p. Furthermore, all the demands are routed such that the maximum load on any single edge of the network is minimized. The load on an edge is the sum of all the traffic that passes through that edge. The problem is NP-hard[136].

Although we will focus on the mathematical formulation as a multicommodity-directed model, many other formulations have been used in network design, depending on the problem context. In the standard formulation, both flow decision variables and design decision variables are modeled explicitly[136–138]. In this case, the linear programming relaxations of multicommodity flow formulations provide lower bounds that are normally weak[138]. To overcome this issue, in ref. 139 the authors remove the flow variables as explicit decisions and embed them within the design variables. Furthermore, in refs. 138,139 the authors also provide a survey of different methods that have been used for network design. For example, in ref. 140 the authors apply Benders decomposition and demonstrate the effectiveness of variable elimination preprocessing and a dual ascent procedure to accelerate the decomposition algorithm. In ref. 136, the authors report on computational experience with a cutting plane algorithm for the above formulation. In ref. 141, they use the column generation technique to obtain near-optimal solutions.

Why is it interesting?. The min–max objective function underlying network design models is linear, and the problem often has a sparse structure. While it is trivial to construct a feasible solution, finding optimal solutions is very difficult. The particular problem presented in this section has been withstanding all attempts to solve it to proven optimality for the last 30 years in its 24-node size.

Problem statement. Given a matrix $T \in \mathbb{N}^{n\times n}$, and an integer $p > 0$, construct a simple directed graph $D = (V, A)$ with vertex set $V = \{1, \ldots, n\}$, where each vertex has in-degree and out-degree equal to p, and simultaneously, in D, route t_{ij} units of flow from vertex i to j, for all $1 \le i \ne j \le n$, such as to minimize the maximum aggregated flow on any arc in D.

We can formulate this network design problem as a multicommodity design problem as follows. For $k, i, j \in \{1, \ldots, n\}$ we define binary variables $x_{ij} \in \{0, 1\}$ which are 1 iff an arc is established between vertex i and j and 0 otherwise. Further, we define non-negative integer variables $f_{kij} \in \mathbb{Z}_+$ that represent the flow of commodity k between vertices i and j, and a suitable big constant M. Then we obtain

$$\min_{x,f,z} z \quad k, i, j \in \{1, \ldots, n\}$$
$$\text{subject to } \sum_{j\ne i} x_{ij} = p \quad \text{for all } i, \tag{4}$$

$$\sum_{j\ne i} x_{ji} = p \text{ for all } i, \tag{5}$$

$$f_{kij} \le M x_{ij} \text{ for all } k, i \ne j, \tag{6}$$

$$\sum_{j\ne i} f_{kij} - \sum_{j\ne i} f_{kji} = t_{ki} \text{ for all } i, k, \tag{7}$$

$$\sum_{k} f_{kij} \le z \text{ for all } i \ne j. \tag{8}$$

Constraints (4) and (5) set the out-degree and in-degree of each vertex to p. Constraint (6) ensures that flow is only allowed on arcs in A. Constraint (7) ensures the flow conservation for each commodity k. (8) ensures that z is greater than the aggregated flow on any arc. Since the objective is to minimize z, (8) will be met with equality at the arc with the highest aggregated flow.

Vehicle routing problem. *Background*. The VRP is the problem of determining the most efficient routes for a fleet of vehicles to deliver goods or services to a set of customers, typically starting and ending at a central depot. This problem is highly relevant for many real-world applications in logistics, transportation and supply chain management. Originally proposed in ref. 142 as 'the truck dispatching problem', many different variants of VRP have emerged since then[143]. Providing a collection of relevant and difficult VRP instances is a complex task because there are numerous variants. Although classical heuristics for VRP are practically very successful, many papers on quantum approaches for VRP problems have been published[144–147]. We include the VRP problem class in this collection to facilitate a comparison with existing work and to explore the capabilities of quantum optimization algorithms for a practically relevant problem.

We limit ourselves to a specific variant of VRP, the CVRP, where the objective is to minimize the aggregated costs across all routes while satisfying the vehicle capacity, effectively limiting the number of goods that each vehicle can carry. This variant, in fact, corresponds to the original problem formulation from ref. 142, where it was presented as a generalization of the TSP[148]. Compared with TSP, VRP involves an additional level of decision-making, as customers need to be not only served but also assigned to vehicles.

Due to the practical and academic relevance of the CVRP, numerous optimization approaches have been developed, and research continues[149]. Exact methods include branch-and-bound and branch-and-cut strategies. A well-known heuristic is the savings algorithm[150], which has been modified and improved in various ways. CVRPs can be further divided into symmetric CVRPs and asymmetric CVRPs, depending on whether the direction of a tour affects its cost. Some solution approaches can only be applied to symmetric CVRPs. Metaheuristics include genetic algorithms, simulated annealing and tabu search. A recent comparison of different heuristics and metaheuristics can be found in ref. 151. Alternative research directions are approaches based on machine learning[152]. More recently, quantum optimization has been explored as a potential strategy. Since naive QUBO formulations for CVRP require many variables[153], hybrid strategies have been used, for example, by separating the problem into a clustering and routing phase. Many hybrid approaches rely on solving the vehicle–customer assignment classically and then solving TSPs using quantum optimization. Successful applications will need to overcome this simplification, as solving TSPs exactly—for example, via Concorde[154]—is unlikely to be outperformed by quantum methods any time soon.

Why is it interesting?. Determining the optimal solution to a VRP is NP-hard[155] and of high practical relevance as it has many direct industrial applications and, hence, the potential for impactful business value. Similar to the network problem from the previous section, it is trivial to find feasible (but not necessarily optimal) solutions. Due to the complexity of the problem, mostly small instances can be solved exactly, but heuristics and metaheuristics are often well suited to tackle real-world instances. This problem class has already received a lot of attention from the quantum community.

Problem statement. Consider a CVRP with $n \ge 1$ customers, where each customer i has a demand $d_i \in \mathbb{N}$ with $i \in \{1, \ldots, n\}$. Furthermore, we assume that there is a homogeneous fleet of $K \ge 1$ vehicles, where each vehicle has a total capacity of $Q \in \mathbb{N}$. Each vehicle starts and ends its tour at the depot. The cost of moving a vehicle from customer i to j is given by $c_{ij} \in \mathbb{N}$ with $i, j \in \{0, \ldots, n\}$ and $i \ne j$, where 'customer 0' is used to represent the depot. With this definition, we presume all-to-all connectivity between the customers (and the depot). While the cost matrix is generally arbitrary, we focus here on a special case relevant to the benchmarking instances we provide, where the costs are based on graph vertex distances in a metric space that satisfies non-negativity (as given by the cost domain) and symmetry (that is, $c_{ij} = c_{ji} \; \forall \, i, j \in \{0, \ldots, n\}$, the condition for the aforementioned symmetric CVRP), which means they also obey the triangle inequality (that is, $c_{ij} \le c_{ik} + c_{kj} \; \forall \, i, j, k \in \{0, \ldots, n\}$). Each customer must be served exactly once by a vehicle, while the total customer demand on a given route must not exceed the capacity limit of the vehicle. The optimization goal is to determine a feasible solution that minimizes the aggregated costs across all routes. This does not necessarily require the use of all K vehicles, that is, some may remain in the depot and hence do not contribute to the costs.

A CVRP can be modeled in many different ways, one of which is the two-index vehicle flow formulation with Miller–Tucker–Zemlin constraints[149]. Following ref. 156, this formulation reads

$$\min_{x,y} \sum_{\substack{i,j=0 \\ i\ne j}}^{n+1} c_{ij} x_{ij} \tag{9}$$

$$\text{subject to } \sum_{\substack{j=1 \\ j\ne i}}^{n+1} x_{ij} = 1 \text{ for all } i \in \{1 \ldots, n\}, \tag{10}$$

$$\sum_{\substack{i=0\\ i\neq h}}^{n} x_{ih} = \sum_{\substack{j=1\\ j\neq h}}^{n+1} x_{hj} \text{ for all } h \in \{1,\dots,n\}, \tag{11}$$

$$\sum_{j=1}^{n} x_{0j} \leq K, \tag{12}$$

$$y_j \geq y_i + d_j x_{ij} - Q(1-x_{ij}) \text{ for all } i \neq j \in \{0,\dots,n+1\}, \tag{13}$$

$$d_i \leq y_i \leq Q, \text{ for all } i \in \{0,\dots,n+1\}, \tag{14}$$

$$x_{ij} \in \{0,1\} \text{ for all } i,j \in \{0,\dots,n+1\}, \tag{15}$$

$$y_i \in \mathbb{N}_0 \text{ for all } i \in \{0,\dots,n+1\}. \tag{16}$$

In this IP, the optimization is done with respect to two types of decision variable: first the binary variables $x \in \{0,1\}^{(n+2)\times(n+2)}$ and second the integer variables $y \in \mathbb{N}_0^{n+2}$. The binary variable x_{ij} takes the value 1 if and only if there is a direct route between customers i and j for $i, j \in \{0, \dots, n+1\}$, where customer 0 is the depot, as introduced above. To simplify the notation, we also symbolically introduce 'customer $n+1$' to represent the same depot, which allows us to distinguish between starting at the depot ($i=0$) and ending at the depot ($i=n+1$), where $c_{j,n+1} \coloneqq c_{0j}$ for all $j \in \{1, \dots, n\}$. Furthermore, the integer variable y_i represents the cumulated demand on the route that visits customer i for $i \in \{0, \dots, n+1\}$, where $d_0 \coloneqq d_{n+1} \coloneqq 0$.

The objective, equation (9), represents the aggregated costs of all routes that need to be minimized. The first set of constraints, equation (10), ensures that each customer is only visited once. The second set of constraints, equation (11), ensures a correct flow of vehicles. The third constraint, inequality (12), ensures that only up to K vehicles can leave the depot. The fourth and fifth set of constraints, inequalities (13) and (14), follow the Miller–Tucker–Zemlin modeling of a TSP[148] and are responsible for ensuring compliance with the vehicle capacities. Using this formulation, a polynomial number of capacity constraints is sufficient (in contrast to an exponential number of constraints with alternative models). Finally, equations (15) and (16) are the integrality constraints.

The problem is often also formulated with real-valued costs and real-valued demands, which only requires the change of y from an integer variable to a continuous variable $y \in \mathbb{R}$, effectively leading to a mixed-integer program. For a more convenient benchmarking set-up, we only provide the integer-based formulation, since it enables us to avoid having to consider machine precision for the instance data, while simultaneously keeping the problem complexity the same.

Topology design. *Background*. Given a graph $G=(V,E)$, we define three possibly competing objectives. First, we may want to focus on order such that the number of vertices $n=|V|$ is maximized. Second, we can target the degree, which relates to the minimization of the maximum vertex degree $d = \max_{v\in V} \delta(v)$. Finally, we may want to minimize the diameter $k = \max_{u,v\in V} \text{length\_of\_shortest\_path}(u,v)$. Fixing two parameters within the set {order, degree, diameter}, and optimizing the third, results in the following optimization problems:

- order degree problem (ODP), minimize the diameter;
- degree diameter problem, maximize the number of vertices;
- order diameter problem, minimize the maximum vertex degree.

Notably, the name of the problem is given by the two fixed parameters. The order diameter problem has received little attention to date. The degree diameter problem has been considered extensively, with the foundations discussed in the literature from the 1960s[157,158]. In this work, we consider the ODP, for which—to our knowledge—an NP-hardness proof is not available in the open literature. The ODP arises in the search for network topologies with low latency[159], and a variety of methodologies for constructing networks with low latency are discussed in the literature[160,161]. We use the ODP variant employed in the Graph Golf competitions[162]: the goal is to solve the ODP, using the average shortest path length as a tie-breaker for graphs with the same diameter, with lower average shortest path length being preferred. This problem is difficult to solve in practice[162].

Why is it interesting?. Several Graph Golf competitions[162] over the years have shown that specifically the ODP is difficult to solve in practice—although the resulting problem formulation can be relatively sparse, and a problem instance is specified by providing just three integers. Furthermore, the variables are typically binary, which facilitates the encoding of the problem in a quantum system. The problem finds application in computer network design[159].

Problem statement. Let $\Delta(s,t)$, $s,t \in V$ be the shortest distance of two vertices in a simple unweighted graph $G=(V,E)$. Given a number $n \in \mathbb{N}$ of vertices, and a maximum vertex degree $d \in \mathbb{N}$ we are looking for a simple graph $G=(V,E)$, $V=\{1,\dots,n\}$, $E \subseteq \{(u,v) \in V \times V | u \neq v\}$ with $\delta(v) \leq d$, for all $v \in V$, that minimizes $\max_{s,t\in V} \Delta(s,t)$. Multiple formulations for the ODP are possible; for an overview, we refer to ref. 163. There is considerable freedom in the choice of how the lengths of the all-pairs shortest paths are computed.

Data availability

The source data for Figs. 1 and 2 are available with this manuscript. All data supporting the findings of this study were generated using different optimization algorithms and are available in the Quantum Optimization Benchmarking Library (https://github.com/ZIB-AOPT/QOBLIB (ref. 5); https://doi.org/10.5281/zenodo.18962381).

Code availability

All code used to generate the results presented in this work can be found in the Quantum Optimization Benchmarking Library (https://github.com/ZIB-AOPT/QOBLIB (ref. 5).

References

1. Abbas A. et al. Challenges and opportunities in quantum optimization. *Nat. Rev. Phys.* **6**, 718–735 (2024).
2. Goemans, M. X. & Williamson, D. P. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM* **42**, 1115–1145 (1995).
3. Khot, S., Kindler, G., Mossel, E. & O'Donnell, R. Optimal inapproximability results for MAX CUT and other 2 variable CSPs?. *SIAM J. Comput.* **37**, 319–357 (2007).
4. Kim, Y. et al. Evidence for the utility of quantum computing before fault tolerance. *Nature* **618**, 500–505 (2023).
5. Quantum Optimization Benchmarking Library (QOBLIB) *zenodo* https://doi.org/10.5281/zenodo.18962381 (2026).
6. Dongarra, J. J., Moler, C. B., Bunch, J. R. & Stewart, G. W. *LINPACK Users' Guide* (SIAM, 1979).
7. Müller, M., Whitney, B., Henschel, R. and Kumaran, K. In *SPEC Benchmarks* 1886–1893 (Springer, 2011).
8. Bartz-Beielstein, T. et al. *Benchmarking in Optimization: Best Practice and Open Issues* (Technology Arts Sciences, TH Köln, 2020).
9. Koch, T. et al. Progress in mathematical programming solvers from 2001 to 2020. *EURO J. Comput. Optim.* **10**, 100031 (2022).
10. Dunning, I., Gupta, S. & Silberholz, J. What works best when? A systematic evaluation of heuristics for Max-Cut and QUBO. *INFORMS J. Comput.* **30**, 608–624 (2018).
11. dimacs2023. *DIMACS Implementation Challenges* (DIMACS, accessed 3 April 2025); http://dimacs.rutgers.edu/programs/challenge

12. Froleyks, N., Heule, M., Iser, M., Järvisalo, M. & Suda, M. SAT competition 2020. *Artif. Intell.* **301**, 103572 (2021).
13. *The International SAT Competition Web Page* https://satcompetition.github.io (accessed 13 May 2026).
14. Gleixner, A. et al. MIPLIB 2017: data-driven compilation of the 6th Mixed-Integer Programming Library. *Math. Program. Comput.* **13**, 443–490 (2021).
15. Mittelmann, H. *Decision Tree for Optimization Software* (Arizona State Univ., accessed 3 April 2025); https://plato.asu.edu/bench.html
16. Proctor, T., Young, K., Baczewski, A. D. & Blume-Kohout, R. Benchmarking quantum computers. *Nat. Rev. Phys.* **7**, 105–118 (2025).
17. Acuaviva, A., Aguirre, D., Peña, R. & Sanz, M. Benchmarking quantum computers: towards a standard performance evaluation approach. *Quantum Sci. Technol.* **11**, 025004 (2026).
18. Lorenz, J. M. et al. Systematic benchmarking of quantum computers: status and recommendations. Preprint at https://arxiv.org/abs/2503.04905 (2025).
19. Magesan, E. et al. Efficient measurement of quantum gate error by interleaved randomized benchmarking. *Phys. Rev. Lett.* **109**, 080505 (2012).
20. McKay, D. C. et al. Benchmarking quantum processor performance at scale. Preprint at https://arxiv.org/abs/2311.05933 (2023).
21. Nation, P. D. et al. Benchmarking the performance of quantum computing software. *Nat. Comput. Sci.* **5**, 427–435 (2025).
22. Wack, A. et al. Quality, speed, and scale: three key attributes to measure the performance of near-term quantum computers. Preprint at https://arxiv.org/abs/2110.14108 (2021).
23. Granet, E. & Dreyer, H. AppQSim: application-oriented benchmarks for Hamiltonian simulation on a quantum computer. *Phys. Rev. Research* **7**, 043146 (2025).
24. Martiel, S., Ayral, T. & Allouche, C. Benchmarking quantum coprocessors in an application-centric, hardware-agnostic, and scalable way. *IEEE Trans. Quantum Eng.* **2**, 1–11 (2021).
25. Lubinski, T. et al. Optimization applications as quantum performance benchmarks. *ACM Trans. Quantum Comput.* **5**, 3 (2024).
26. Tomesh, T. et al. SupermarQ: a scalable quantum benchmark suite. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* 587–603 (IEEE, 2022).
27. Santra, G. C., Jendrzejewski, F., Hauke, P. & Egger, D. J. Squeezing and quantum approximate optimization. *Phys. Rev. A* **109**, 012413 (2024).
28. Finžgar, J. R., Ross, P., Hölscher, L., Klepsch, J. & Luckow, A. QUARK: a framework for quantum computing application benchmarking. In *2022 IEEE International Conference on Quantum Computing and Engineering (QCE)* 226–237 (IEEE, 2022).
29. McGeoch, C. How not to fool the masses when giving performance results for quantum computers. Preprint at https://arxiv.org/abs/2411.08860 (2024).
30. Bernal Neira, D. E. et al. Benchmarking the operation of quantum heuristics and Ising machines: scoring parameter setting strategies on optimization applications. *Quantum Mach. Intell.* **7**, 86 (2025).
31. Lall, D. et al. A review and collection of metrics and benchmarks for quantum computers: definitions, methodologies and software. Preprint at https://arxiv.org/abs/2502.06717 (2025).
32. Wu, D. et al. Variational benchmarks for quantum many-body problems. *Science* **386**, 296–301 (2024).
33. Karp, R. M. Reducibility among combinatorial problems. In *Complexity of Computer Computations: Proc. Symposium on the Complexity of Computer Computations* (eds Miller, R. E. et al.) 85–103 (Springer, 1972).
34. Agarwal, P. K., Van Kreveld, M. & Suri, S. Label placement by maximum independent set in rectangles. *Comput. Geom.* **11**, 209–218 (1998).
35. Hossain, A. et al. Automated design of thousands of nonrepetitive parts for engineering stable genetic systems. *Nat. Biotechnol.* **38**, 1466–1475 (2020).
36. Wurtz, J. et al. Industry applications of neutral-atom quantum computing solving independent set problems. Preprint at https://arxiv.org/abs/2205.08500 (2022).
37. Xiao, M. & Nagamochi, H. Exact algorithms for maximum independent set. *Inf. Comput.* **255**, 126–146 (2017).
38. Bazgan, C., Escoffier, B. & Paschos, V. T. Completeness in standard and differential approximation classes: Poly-(D)APX- and (D)PTAS-completeness. *Theor. Comput. Sci.* **339**, 272–292 (2005).
39. Clark, B. N., Colbourn, C. J. & Johnson, D. S. Unit disk graphs. *Discret. Math.* **86**, 165–177 (1990).
40. Ebadi, S. et al. Quantum optimization of maximum independent set using Rydberg atom arrays. *Science* **376**, 1209–1215 (2022).
41. Marathe, M. V., Breu, H., Hunt, H. B. III, Ravi, S. S. & Rosenkrantz, D. J. Simple heuristics for unit disk graphs. *Networks* **25**, 59–68 (1995).
42. Das, G. K., da Fonseca, G. D. & Jallu, R. K. Efficient independent set approximation in unit disk graphs. *Discret. Appl. Math.* **280**, 63–70 (2020).
43. Sloane, N. J. A. *Challenge Problems: Independent Sets in Graphs* (OEIS Foundation, accessed 25 February 2025); https://oeis.org/A265032/a265032.html
44. Rossi, R., & Ahmed, N. The network data repository with interactive graph analytics and visualization. In *Proc. AAAI Conference on Artificial Intelligence*, Vol. 29 (2015); https://doi.org/10.1609/aaai.v29i1.9277
45. Koch, T., Martin, A. & Voß, S. *SteinLib: an Updated Library on Steiner Tree Problems in Graphs* Technical Report ZIB-Report 00-37 (Konrad-Zuse-Zentrum für Informationstechnik Berlin, 2000).
46. Farhi, E., Goldstone, J. & Gutmann, S. A quantum approximate optimization algorithm. Preprint at https://arxiv.org/abs/1411.4028 (2014).
47. Best practices in quantum optimization. *GitHub* https://github.com/qiskit-community/qopt-best-practices (2025).
48. Egger, D. J., Mareček, J. & Woerner, S. Warm-starting quantum optimization. *Quantum* **5**, 479 (2021).
49. Alessandroni, E., Ramos-Calderer, S., Roth, I., Traversi, E. & Aolita, L. Alleviating the quantum Big-*M* problem. Preprint at https://arxiv.org/abs/2307.10379 (2024).
50. Virtanen, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nat. Methods* **17**, 261–272 (2020).
51. *IBM Quantum Platform* (IBM Quantum, accessed 17 February 2025); https://quantum.ibm.com
52. Weidenfeller, J. et al. Scaling of the quantum approximate optimization algorithm on superconducting qubit based hardware. *Quantum* **6**, 870 (2022).
53. Bennett, C. H., Bernstein, E., Brassard, G. & Vazirani, U. Strengths and weaknesses of quantum computing. *SIAM J. Comput.* **26**, 1510–1523 (1997).
54. de Wolf, R. Quantum computing: lecture notes. Preprint at https://arxiv.org/abs/1907.09415 (2023).
55. Durr, C. & Hoyer, P. A quantum algorithm for finding the minimum. Preprint at https://arxiv.org/abs/quant-ph/9607014 (1999).
56. Gilliam, A., Woerner, S. & Gonciulea, C. Grover adaptive search for constrained polynomial binary optimization. *Quantum* **5**, 428 (2021).
57. Jordan, S. P. et al. Optimization by decoded quantum interferometry. *Nature* **646**, 831–836 (2025).
58. Schrijver, A. *Combinatorial Optimization* (Springer, 2003).

59. Achterberg, T., Koch, T. & Tuchscherer, A. *On the Effects of Minor Changes in Model Formulations* Technical Report 08-29 (ZIB, 2008).
60. Lucas, A. Ising formulations of many NP problems. *Front. Phys.* **2**, 5 (2014).
61. *QUBO++ with ABS2 GPU QUBO Solver* (Hiroshima University, accessed 31 March 2025); https://abs2.cs.hiroshima-u.ac.jp/home
62. Pelofske, E., Bärtschi, A. & Eidenbenz, S. Short-depth QAOA circuits and quantum annealing on higher-order Ising models. *npj Quantum Inf.* **10**, 30 (2024).
63. Anthony, M., Boros, E., Crama, Y. & Gruber, A. Quadratic reformulations of nonlinear binary optimization problems. *Math. Program.* **162**, 115–144 (2017).
64. Glover, F., Kochenberger, G. & Du, Y. A tutorial on formulating and using QUBO models. *4OR* **17**, 335–371 (2019).
65. Xavier, P. M. et al. Disjunctive programming meets QUBO. In *Computer Aided Chemical Engineering* Vol. 53 (eds Manenti, F. & Reklaitis, G. V.) 3433–3438 (Elsevier, 2024).
66. Berthold, T. Measuring the impact of primal heuristics. *Oper. Res. Lett.* **41**, 611–614 (2013).
67. Kool, W., van Hoof, H. & Welling, M. Attention, learn to solve routing problems! In *International Conference on Learning Representations (ICLR 2019)* (2019).
68. Berto, F. et al. RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark. In *KDD 2025 — Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining* 5278–5289 (ACM, 2025).
69. Mandelbaum, R., Corcoles, A. D. & Gambetta, J. *IBM's Big Bet on the Quantum-Centric Supercomputer* https://spectrum.ieee.org/ibm-quantum-computer-2668978269 (2024).
70. Javadi-Abhari, A. et al. Quantum computing with Qiskit. Preprint at https://arxiv.org/abs/2405.08810 (2024).
71. Sciorilli, M. et al. Towards large-scale quantum optimization solvers with few qubits. *Nat. Commun.* **16**, 476 (2025).
72. Gérard Cornuéjols, M. D. Class of hard small 0–1 programs. *INFORMS J. Comput.* **11**, 205–210 (1999).
73. Garey, M. R. & Johnson, D. S. *Computers and Intractability: a Guide to the Theory of NP-Completeness* (Freeman, 1979).
74. Schroeppel, R. & Shamir, A. A $t = O(2^{n/2})$, $s = O(2^{n/4})$ algorithm for certain NP-complete problems. *SIAM J. Comput.* **10**, 456–464 (1981).
75. Wassermann, A. Attacking the market split problem with lattice point enumeration. *J. Comb. Optim.* **6**, 5–16 (2002).
76. Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K. & Smeltink, J. W. Market split and basis reduction: towards a solution of the Cornuéjols–Dawande instances. *INFORMS J. Comput.* **12**, 192–202 (2000).
77. Boehmer, A. Binary pulse compression codes. *IEEE Trans. Inf. Theory* **13**, 156–167 (1967).
78. Mertens, S. & Bessenrodt, C. On the ground states of the Bernasconi model. *J. Phys. A* **31**, 3731–3750 (1998).
79. Elloumi, S., Lambert, A. & Lazare, A. Solving unconstrained 0–1 polynomial programs through quadratic convex reformulation. *J. Glob. Optim.* **80**, 231–248 (2021).
80. *MINLPLib: a Library of Mixed-Integer and Continuous Nonlinear Programming Instances* (accessed 3 April 2025); https://www.minlplib.org/index.html
81. Packebusch, T. & Mertens, S. Low autocorrelation binary sequences. *J. Phys. A* **49**, 165001 (2016).
82. Gallardo, J. E., Cotta, C. & Fernández, A. J. Finding low autocorrelation binary sequences with memetic algorithms. *Appl. Soft Comput.* **9**, 1252–1262 (2009).
83. Shaydulin, R. et al. Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem. *Sci. Adv.* **10**, eadm6761 (2024).
84. Zhang, Z., Shen, J., Kumar, N. & Pistoia, M. New improvements in solving large LABS instances using massively parallelizable memetic tabu search. Preprint at https://arxiv.org/abs/2504.00987 (2025).
85. Dufossé, F. & Uçar, B. Notes on Birkhoff–von Neumann decomposition of doubly stochastic matrices. *Linear Algebra Its Appl.* **497**, 108–115 (2016).
86. Birkhoff, G. Tres observaciones sobre el algebra lineal. *Univ. Nac. Tucuman A* **5**, 147–154 (1946).
87. Brualdi, R. A. Notes on the Birkhoff algorithm for doubly stochastic matrices. *Can. Math. Bull.* **25**, 191–199 (1982).
88. Marcus, M. & Ree, R. Diagonals of doubly stochastic matrices. *Q. J. Math.* **10**, 296–302 (1959).
89. Kulkarni, J., Lee, E. & Singh, M. in *Integer Programming and Combinatorial Optimization* (eds Eisenbrand, F. & Koenemann, J.) 343–354 (Springer, 2017).
90. Chang, C. S., Chen, W. J. & Huang, H. Y. Birkhoff–von Neumann input buffered crossbar switches. In *Proc. IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies* Vol. 3 1614–1623 (IEEE, 2000).
91. Ye, T., Zhang, J., Lee, T. T. & Hu, W. Deflection-compensated Birkhoff–von-Neumann switches. *IEEE/ACM Trans. Netw.* **25**, 879–895 (2017).
92. Liu, H. et al. Scheduling techniques for hybrid circuit/packet networks. In *Proc. 11th ACM Conference on Emerging Networking Experiments and Technologies* 41 (Association for Computing Machinery, 2015).
93. O'Meara, C., Fernández-Campoamor, M., Cortiana, G. & Bernabé-Moreno, J. Quantum software architecture blueprints for the Cloud: overview and application to peer-2-peer energy trading. In *2023 IEEE Conference on Technologies for Sustainability (SusTech)* 191–198 (IEEE, 2023).
94. Hendrych, D., Troppens, H., Besançon, M. & Pokutta, S. Convex mixed-integer optimization with Frank–Wolfe methods. *Math. Prog. Comp.* **17**, 731–757 (2025).
95. Combettes, C. W. & Pokutta, S. Revisiting the approximate Carathéodory problem via the Frank–Wolfe algorithm. *Math. Program.* **197**, 191–214 (2023).
96. Valls, V., Iosifidis, G. & Tassiulas, L. Birkhoff's decomposition revisited: sparse scheduling for high-speed circuit switches. *IEEE/ACM Trans. Netw.* **29**, 2399–2412 (2021).
97. Mariella, N. et al. Quantum theory and application of contextual optimal transport. In *Proc. 41st International Conference on Machine Learning* Vol. 235, 34822–34845 (2024).
98. Arora, S. Polynomial time approximation schemes for Euclidean TSP and other geometric problems. In *Proc. 37th Conference on Foundations of Computer Science* 2–11 (IEEE, 1996).
99. Hakimi, S. L. Steiner's problem in graphs and its implications. *Networks* **1**, 113–133 (1971).
100. Grötschel, M., Martin, A. & Weismantel, R. The Steiner tree packing problem in VLSI design. *Math. Program.* **78**, 265–281 (1997).
101. Grötschel, M., Martin, A. & Weismantel, R. Packing Steiner trees: polyhedral investigations. *Math. Program.* **72**, 101–123 (1996).
102. Grötschel, M., Martin, A. & Weismantel, R. Packing Steiner trees: separation algorithms. *SIAM J. Discret. Math.* **9**, 233–257 (1996).
103. Jain, K., Mahdian, M. & Salavatipour, M. R. Packing Steiner trees. In *SODA '03: Proc. Fourteenth Annual ACM-SIAM Symposium on Discrete Algorithms* Vol. 3 266–274 (Society for Industrial and Applied Mathematics, 2003).
104. Freund, J. E. Round robin mathematics. *Am. Math. Mon.* **63**, 112–114 (1956).
105. Kaski, P. & Östergård, P. R. J. There are 1,132,835,421,602,062,347 nonisomorphic one-factorizations of K_{14}. *J. Comb. Des.* **17**, 147–159 (2009).

106. van Doornmalen, J., Hojny, C., Lambers, R. & Spieksma, F. C. R. Integer programming models for round robin tournaments. *Eur. J. Oper. Res.* **310**, 24–33 (2023).
107. Nemhauser, G. L. & Trick, M. A. Scheduling a major college basketball conference. *Oper. Res.* **46**, 1–8 (1998).
108. Van Bulck, D. & Goossens, D. A traditional Benders' approach to sports timetabling. *Eur. J. Oper. Res.* **307**, 813–826 (2023).
109. Van Bulck, D. & Goossens, D. The international timetabling competition on sports timetabling (ITC2021). *Eur. J. Oper. Res.* **308**, 1249–1267 (2023).
110. Van Bulck, D. et al. Which algorithm to select in sports timetabling? *Eur. J. Oper. Res.* **318**, 575–591 (2024).
111. Markowitz, H. Portfolio selection. *J. Financ.* **7**, 77–91 (1952).
112. Kolm, P. N., Tütüncü, R. & Fabozzi, F. J. 60 years of portfolio optimization: practical challenges and current trends. *Eur. J. Oper. Res.* **234**, 356–371 (2014).
113. Haff, L. R. Empirical Bayes estimation of the multivariate normal covariance matrix. *Ann. Stat.* **8**, 586–597 (1980).
114. Ben-Tal, A. & Nemirovski, A. Robust solutions of uncertain linear programs. *Oper. Res. Lett.* **25**, 1–13 (1999).
115. Black, F. & Litterman, R. Global portfolio optimization. *Financ. Anal. J.* **48**, 28–43 (1992).
116. Michaud, R. O. & Ma, T. Efficient asset management: a practical guide to stock portfolio optimization and asset allocation. *Rev. Financ. Stud.* **14**, 901–904 (2015).
117. Merton, R. C. Lifetime portfolio selection under uncertainty: the continuous-time case. *Rev. Econ. Stat.* **51**, 247–257 (1969).
118. DeMiguel, V., Garlappi, L. & Uppal, R. Optimal versus naive diversification: how inefficient is the 1/*n* portfolio strategy?. *Rev. Financ. Stud.* **22**, 1915–1953 (2007).
119. López de Prado, M. Building diversified portfolios that outperform out of sample. *J. Portf. Manag.* **42**, 59–69 (2016).
120. Gava, J. & Turc, J. The properties of alpha risk parity portfolios. *Entropy* **24**, 1631 (2022).
121. Almgren, R. & Chriss, N. Optimal execution of portfolio transactions. *J. Risk* **3**, 5–39 (2000).
122. Palczewski, J., Poulsen, R., Schenk-Hoppé, K. R. & Wang, H. Dynamic portfolio optimization with transaction costs and state-dependent drift. *Eur. J. Oper. Res.* **243**, 921–931 (2015).
123. Kolm, P. N. and Ritter, G. Modern perspectives on reinforcement learning in finance. *SSRN Electron. J.* (2019).
124. Brown, D. B. & Smith, J. E. Dynamic portfolio optimization with transaction costs: heuristics and dual bounds. *Manag. Sci.* **57**, 1752–1770 (2011).
125. Constantinides, G. M. Multiperiod consumption and investment behavior with convex transactions costs. *Manag. Sci.* **25**, 1127–1137 (1979).
126. Kerenidis, I., Prakash, A. & Szilágyi, D. Quantum algorithms for portfolio optimization. In *Proc. 1st ACM Conference on Advances in Financial Technologies* 147–155 (Association for Computing Machinery, 2019).
127. Rosenberg, G. et al. Solving the optimal trading trajectory problem using a quantum annealer. *IEEE J. Sel. Top. Signal Process.* **10**, 1053–1060 (2016).
128. Lim, D. & Rebentrost, P. A quantum online portfolio optimization algorithm. *Quantum Inf. Process.* **23**, 63 (2024).
129. Brandhofer, S. et al. Benchmarking the performance of portfolio optimization with QAOA. *Quantum Inf. Process.* **22**, 25 (2022).
130. Slate, N., Matwiejew, E., Marsh, S. & Wang, J. B. Quantum walk-based portfolio optimisation. *Quantum* **5**, 513 (2021).
131. Mugel, S. et al. Dynamic portfolio optimization with real datasets using quantum processors and quantum-inspired tensor networks. *Phys. Rev. Res.* **4**, 013006 (2022).
132. Bienstock, D. Computational study of a family of mixed-integer quadratic programming problems. *Math. Program.* **74**, 121–140 (1996).
133. Gavish, B. Topological design of telecommunication networks—local access design methods. *Ann. Oper. Res.* **33**, 17–71 (1991).
134. Magnanti, T. L. & Wong, R. T. Network design and transportation planning: models and algorithms. *Transp. Sci.* **18**, 1–55 (1984).
135. Saberi, R. et al. Power distribution network expansion planning to improve resilience. *IET Gener. Transm. Distrib.* **17**, 4701–4716 (2023).
136. Bienstock, D. & Günlük, O. Computational experience with a difficult mixedinteger multicommodity flow problem. *Math. Program.* **68**, 213–237 (1995).
137. Bienstock, D., Chopra, S., Günlük, O. & Tsai, C.-Y. Minimum cost capacity installation for multicommodity network flows. *Math. Program.* **81**, 177–199 (1998).
138. Gendron, B., Crainic, T. G. & Frangioni, A. in *Telecommunications Network Planning* (eds Sansò, B. & Soriano, P.) 1–19 (Springer, 1999).
139. Armacost, A. P., Barnhart, C. & Ware, K. A. Composite variable formulations for express shipment service network design. *Transp. Sci.* **36**, 1–20 (2002).
140. Magnanti, T. L., Mireault, P. & Wong, R. T. *Tailoring Benders Decomposition for Uncapacitated Network Design* (Springer, 1986).
141. Barnhart, C. & Schneur, R. R. Air network design for express shipment service. *Oper. Res.* **44**, 852–863 (1996).
142. Dantzig, G. B. & Ramser, J. H. The truck dispatching problem. *Manag. Sci.* **6**, 80–91 (1959).
143. Helsgaun, K. *An Extension of the Lin–Kernighan–Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems* Technical Report (Roskilde Univ., 2017).
144. Xie, N. et al. A feasibility-preserved quantum approximate solver for the capacitated vehicle routing problem. *Quantum Inf. Process.* **23**, 291 (2024).
145. Mohanty, N., Behera, B. K. & Ferrie, C. Analysis of the vehicle routing problem solved via hybrid quantum algorithms in the presence of noisy channels. *IEEE Trans. Quantum Eng.* **4** (2023).
146. Leonidas, I. D. et al. Qubit efficient quantum algorithms for the vehicle routing problem on noisy intermediate-scale quantum processors. *Adv. Quantum Technol.* **7**, 2300309 (2024).
147. Fitzek, D., Ghandriz, T., Laine, L., Granath, M. & Kockum, A. F. Applying quantum approximate optimization to the heterogeneous vehicle routing problem. *Sci. Rep.* **14**, 25415 (2021).
148. Miller, C. E., Tucker, A. W. & Zemlin, R. A. Integer programming formulation of traveling salesman problems. *J. ACM* **7**, 326–329 (1960).
149. Sidorov, K. & Morozov, A. A review of approaches to modeling applied vehicle routing problems. Preprint at https://arxiv.org/abs/2105.10950 (2021).
150. Clarke, G. & Wright, J. W. Scheduling of vehicles from a central depot to a number of delivery points. *Oper. Res.* **12**, 568–581 (1964).
151. Muriyatmoko, D., Djunaidy, A. & Muklason, A. Heuristics and metaheuristics for solving capacitated vehicle routing problem: an algorithm comparison. *Procedia Comput. Sci.* **234**, 494–501 (2024).
152. El Jaouhari, M., Bencheikh, G. & Bencheikh, G. Exploring the capacitated vehicle routing problem using the power of machine learning: a literature review. In *Proc. 7th International Conference on Logistics Operations Management, GOL'24* (eds Benadada, Y. et al.) 68–80 (Springer Nature, 2024).
153. Harikrishnakumar, R., Nannapaneni, S., Nguyen, N. H., Steck, J. E. & Behrman, E. C. A quantum annealing approach for dynamic multi-depot capacitated vehicle routing problem. Preprint at https://arxiv.org/abs/2005.12478 (2020).
154. Applegate, D., Bixby, R., Chvátal, V. & Cook, W. Implementing the Dantzig–Fulkerson–Johnson algorithm for large traveling salesman problems. *Math. Program.* **97**, 91–153 (2003).

155. Lenstra, J. K. & Kan, A. H. G. R. Complexity of vehicle routing and scheduling problems. *Networks* **11**, 221–227 (1981).
156. Munari, P., Dollevoet, T. & Spliet, R. A generalized formulation for vehicle routing problems. Preprint at https://arxiv.org/abs/1606.01935 (2017).
157. Hoffman, A. J. & Singleton, R. R. On Moore graphs with diameters 2 and 3. *IBM J. Res. Dev.* **4**, 497–504 (1960).
158. Cerf, V. G., Cowan, D. D., Mullin, R. C. & Stanton, R. G. A lower bound on the average shortest path length in regular graphs. *Networks* **4**, 335–342 (1974).
159. Shin, J. Y., Wong, B. & Sirer, E. G. Small-world datacenters. In *Proc. 2nd ACM Symposium on Cloud Computing* 2 (Association for Computing Machinery, 2011).
160. Besta M. & Hoefler, T. Slim Fly: a cost effective low-diameter network topology. In *SC'14: Proc. International Conference for High Performance Computing, Networking, Storage and Analysis* 348–359 (IEEE, 2014).
161. Singla, A., Hong, C. Y., Popa, L. & Godfrey, P. B. Jellyfish: networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)* 225–238 (USENIX Association, 2012).
162. Koibuchi, M. et al. *Graph Golf: the Order/Degree Problem Competition, 2015* (accessed 16 September 2024); https://research.nii.ac.jp/graphgolf/
163. Waniek, R. *Graph Golf: Betrachtung des Order-Degree-Problems*. Master's thesis, Technische Univ. Berlin (2021).
164. QAOA Bench. Independent set benchmarking. *GitHub* https://github.com/eggerdj/independent_set_benchmarking (2025).
165. QiskitOpt. Qiskit Ecosystem: Qiskit Optimization. *GitHub* https://github.com/qiskit-community/qiskit-optimization (2025).
166. Koch, T. *Rapid Mathematical Programming*. PhD thesis, Technische Univ. Berlin, ZIB-Report 04-58 (2004).

Acknowledgements

We thank K. Aardal at TU-Delft for providing results for the Market Split instances using lattice point enumeration, and N.-C. Kempke at ZIB for providing results using GPU-accelerated Schroeppel–Shamir enumeration. We thank S. Designolle at the Zuse Institute Berlin for discussions on the blended FW algorithm used in the classical benchmarking of the minimum Birkhoff decomposition problem. We also thank J. Gacon for discussions and a review, and T. Imamichi for assistance with software issues, sharing his insights and providing feedback. Additionally, we are grateful to A. Wack and P. Nation for conversations on benchmarking quantum algorithms and for their feedback on this work. Part of the work for this article has been conducted in the Research Campus MODAL funded by the German Federal Ministry of Education and Research (BMBF) (fund numbers 05M14ZAM, 05M20ZBM, 05M2025). G.N. is supported by ONR award N000142312585. Y.C. acknowledges the support from the Quantum Engineering Program project A-8000339-01-00, from the Ministry of Education Singapore under project A-8000014-00-00 and from the National Research Foundation Singapore under its Industry Alignment Fund—Pre-positioning (IAF-PP) Funding Initiative and the Monetary Authority of Singapore under A-0003504-17-00. D.V.B. acknowledges the support of the Research Foundation Flanders (FWO) (23AXE35N).

Author contributions

This Resource was written as part of the Quantum Optimization Working Group initiated in July 2023 by IBM Quantum and its partners. T. Koch, conceptualization, software, investigation, resources, data curation, writing—original draft, writing—review and editing, supervision; D.E.B.N., conceptualization, writing—original draft, writing—review and editing; Y.C., conceptualization, investigation, modeling, resources, writing—original draft; G.C., conceptualization, writing—review and editing; D.J.E., investigation, data curation, writing—original draft; R. Heese, investigation, resources, writing—original draft; N.N.H., investigation, resources, writing—original draft; A.G.C., investigation, resources, writing—original draft; R. Huang, software, data curation; T.I., writing—original draft; T. Kleinert, resources, writing—original draft; P.M.X., writing—original draft, investigation, software; N.M., investigation, data curation, writing—original draft; J.A.M.-B., writing—review and editing; K.N., software, investigation; G.N., conceptualization, methodology, software, investigation, resources, data curation, writing—original draft, writing—review and editing; C.O'M., conceptualization, writing—review and editing; J.P., investigation, resources; M.P., writing—original draft; A.R., writing—original draft, investigation; M.S., software, investigation, resources, data curation; N.S., software, investigation; M.T., investigation; V.V., methodology, software, investigation, resources, data curation, writing—original draft; D.V.B., software, investigation, data curation; S.W., conceptualization, writing—original draft, writing—review and editing, supervision; C.Z., conceptualization, writing—original draft, writing—review and editing, supervision.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at https://doi.org/10.1038/s43588-026-00991-1.

Correspondence and requests for materials should be addressed to Thorsten Koch, Stefan Woerner or Christa Zoufal.

Peer review information *Nature Computational Science* thanks Dong-Ling Deng, Jianxin Chen and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. Primary Handling Editor: Jie Pan, in collaboration with the *Nature Computational Science* team.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© IBM and its affiliates 2026